



МИНОБРНАУКИ РОССИИ
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«МИРЭА – Российский технологический университет»
РТУ МИРЭА

Институт информационных технологий (ИТ)
Кафедра инструментального и прикладного программного обеспечения (ИиППО)

ЗАДАНИЕ
на выполнение курсовой работы

по дисциплине: Проектирование и разработка клиент-серверных приложений
по профилю: Разработка программных продуктов и проектирование информационных систем
направления профессиональной подготовки: Программная инженерия (09.03.04)

Студент: Мусатов Иван Алексеевич

Группа: ИКБО-11-23

Срок представления к защите: 18.05.2026

Руководитель: ст. преп. Сеницын Анатолий Васильевич

Тема: «Клиент-серверное приложение для создания и распространения интерактивных образовательных материалов»

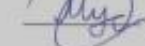
Исходные данные: методология двенадцати факторов, Git, Нормативный документ: инструкция по организации и проведению курсового проектирования СМКО МИРЭА 7.5.1/04.И.05-18.

Перечень вопросов, подлежащих разработке, и обязательного графического материала: 1. Провести анализ предметной области для выбранной темы. 2. Выбрать клиент-серверную архитектуру для разрабатываемого приложения и дать её детальное описание с помощью UML. 3. Выбрать программный стек для реализации фуллстек CRUD приложения. 4. Разработать клиентскую и серверную части приложения, реализовать авторизацию и аутентификацию пользователя, обеспечить работу с базой данных, заполнить тестовыми данными, валидировать ролевою модель на некорректные данные. 5. Провести фаззинг-тестирование. 6. Разместить исходный код клиент-серверного приложения в системе контроля версий с наличием Dockerfile и описанием структуры проекта в readme файле. 7. Развернуть клиент-серверное приложение в облаке. 8. Разработать презентацию с графическими материалами.

Руководителем произведён инструктаж по технике безопасности, противопожарной технике и правилам внутреннего распорядка.

Зав. кафедрой ИиППО:  / Болбаков Р. Г. /, «18» февраля 2026 г.

Задание на КР выдал:  / Сеницын А. В. /, «18» февраля 2026 г.

Задание на КР получил:  / Мусатов И. А. /, «18» февраля 2026 г.

АННОТАЦИЯ

Отчет 77 стр., 49 рисунков, 13 таблиц, 11 источников.

Ключевые слова: КЛИЕНТ-СЕРВЕРНОЕ ПРИЛОЖЕНИЕ, ИНТЕРАКТИВНЫЙ ДОКУМЕНТ, ФОРМАТ ФАЙЛА, ВЕРИФИКАЦИЯ, WEBASSEMBLY, HMAC-SHA256, REACT, SPRING BOOT.

Объектом исследования является процесс создания, публикации и верификации интерактивных учебно-информационных материалов как самостоятельных цифровых артефактов.

Цель работы – проектирование и разработка клиент-серверного приложения для создания, публикации и распространения интерактивных документов, а также формализация спецификации формата документов.

Методы исследования и реализации включают анализ предметной области, функциональное и архитектурное проектирование с использованием нотаций IDEF0, C4 и UML, реализацию бэкенда на Spring Boot, клиентской части на React с парсером на Rust (WebAssembly), модульное и фаззинг-тестирование, контейнеризацию (Docker) и развертывание в облачной среде.

Результатом работы является программный продукт – клиент-серверная платформа Doseum, предоставляющая пользователям возможности по созданию, публикации, верификации и распространению интерактивных документов в формате .doseo, а также пакет документации, включающий спецификацию формата и руководство по развертыванию.

Новизна работы заключается в разработке открытого самодостаточного формата .doseo, сочетающего высокую интерактивность с полной переносимостью и возможностью криптографической верификации, а также в выносе тяжелых операций парсинга на клиентскую сторону с помощью WebAssembly.

Область применения результатов включает создание интерактивных инструкций, гайдов, методических материалов, технической документации, а также образовательных и корпоративных материалов, требующих переносимости, долгосрочной сохранности и подтверждения авторства.

СОДЕРЖАНИЕ

ТЕРМИНЫ И ОПРЕДЕЛЕНИЯ.....	6
ПЕРЕЧЕНЬ СОКРАЩЕНИЙ И ОБОЗНАЧЕНИЙ	7
ВВЕДЕНИЕ	8
1 АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ	10
1.1 Обследование предметной области.....	10
1.2 Анализ существующих решений.....	11
1.3 Выявленные проблемы и ограничения	14
1.4 Цели и назначение разработки	15
1.5 Сценарии использования системы	16
1.6 Формирование требований к продукту	19
1.6.1 Функциональные требования	19
1.6.2 Нефункциональные требования	21
1.6.3 Ограничения системы.....	23
1.6.4 Требования к формату интерактивного документа.....	24
1.7 Итоги анализа предметной области	25
2 ПРОЕКТИРОВАНИЕ СИСТЕМЫ	26
2.1 Концепция продукта	26
2.1.1 Общая характеристика продукта.....	26
2.1.2 Особенности и принципы реализации.....	27
2.1.3 Целевая аудитория.....	27
2.2 Функциональная модель системы	28
2.3 Обоснование архитектурных решений	31
2.3.1 Обоснование выбора вида архитектуры.....	31
2.3.2 Обоснование выбора типа клиентской части	32
2.3.3 Обоснование архитектурного стиля серверной части	32
2.4 Проектирование архитектуры системы	33
2.4.1 Схема контейнеров системы.....	34
2.4.2 Структура компонентов серверной части	35
2.4.3 Архитектура развертывания системы.....	36
2.4.4 Сценарии взаимодействия компонентов системы	38

2.5 Проектирование формата интерактивного документа.....	39
2.6 Проектирование баз и хранилищ данных.....	41
2.7 Проектирование программного интерфейса	42
2.8 Проектирование пользовательского интерфейса	44
2.9 Обоснование выбора инструментария.....	46
2.9.1 Выбор технологий серверной части	46
2.9.2 Выбор технологий клиентской части	47
2.9.3 Выбор инфраструктурных решений	48
3 РЕАЛИЗАЦИЯ ПРОГРАММНОГО ОБЕСПЕЧЕНИЯ	49
3.1 Организация исходного кода и структура проекта	49
3.2 Реализация слоя данных.....	51
3.3 Реализация модуля аутентификации и авторизации	54
3.4 Реализация модуля управления документами	55
3.5 Реализация парсера формата документа	56
3.6 Реализация клиентской части	58
3.7 Конфигурация сетевого взаимодействия.....	60
3.8 Результаты разработки	62
4 ТЕСТИРОВАНИЕ И РАЗВЕРТЫВАНИЕ	64
4.1 Тестирование программного обеспечения	64
4.1.1 Модульное тестирование	64
4.1.2 Фаззинг-тестирование	67
4.2 Верификация продукта.....	68
4.3 Контейнеризация системы	70
4.4 Документирование продукта	72
4.4.1 Спецификация формата интерактивного документа	72
4.4.2 Руководство по развертыванию	73
4.5 Настройка CI.....	74
4.6 Развертывание в облачной среде.....	75
ЗАКЛЮЧЕНИЕ	77
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	79

ТЕРМИНЫ И ОПРЕДЕЛЕНИЯ

В настоящем отчете применяются следующие термины с соответствующими определениями.

.doceo	– файловый формат интерактивного документа, разработанный в рамках работы
Doseum	– наименование разрабатываемой платформы
Self-contained	– свойство документа, означающее, что он содержит все необходимые ресурсы внутри себя
Верификация	– процесс проверки подписи документа для подтверждения его целостности и подлинности
Избранное	– список публикаций, сохраненных пользователем для быстрого доступа
Интерактивный документ	– документ, содержащий элементы для активного взаимодействия с читателем
Каталог	– раздел платформы со списком опубликованных документов, доступных для просмотра и поиска
Подпись документа	– механизм HMAC-SHA256, обеспечивающий целостность и подтверждение авторства документа
Публикация	– документ, опубликованный в каталоге и доступный для просмотра читателями
Редактор	– компонент платформы для создания и редактирования интерактивных документов
Ридер	– компонент платформы для просмотра интерактивных документов
Черновик	– документ, доступный только автору для редактирования

ПЕРЕЧЕНЬ СОКРАЩЕНИЙ И ОБОЗНАЧЕНИЙ

В настоящем отчете применяются следующие сокращения и обозначения.

API	– программный интерфейс приложения (Application Programming Interface)
CORS	– механизм междоменного обмена ресурсами, позволяющий веб-странице запрашивать ресурсы с другого домена (Cross-Origin Resource Sharing)
ERD	– диаграмма «сущность-связь» (Entity-Relationship Diagram)
HMAC	– код аутентификации сообщений на основе хэш-функции (Hash-based Message Authentication Code)
IDEF0	– методология функционального моделирования (Integration Definition for Function Modeling)
JWT	– компактный и безопасный способ передачи утверждений в формате JSON между сторонами (JSON Web Token)
REST	– архитектурный стиль для построения веб-API, использующий стандартные HTTP-методы (Representational State Transfer)
SPA	– одностраничное приложение, загружающее всю страницу однократно и динамически обновляющее содержимое без перезагрузки (Single Page Application)
TTL	– время жизни данных, по истечении которого запись автоматически удаляется (Time To Live)
WASM	– бинарный формат инструкций для высокопроизводительного выполнения кода в браузере (WebAssembly)

ВВЕДЕНИЕ

С древнейших времен передача знаний от человека к человеку является основой развития цивилизации. От наскальных рисунков до цифровых документов форма фиксации знания всегда определяла глубину его усвоения. Сегодня информация стала общедоступной, однако ее избыточность не гарантирует качества восприятия. Статические текстовые файлы и видеолекции оставляют читателя пассивным наблюдателем. Именно в этом разрыве между объемом информации и эффективностью ее усвоения видится ключевая проблема современной образовательной коммуникации.

Педагогические исследования подтверждают, что интерактивность и вовлеченность пользователя улучшают восприятие материала [1]. Однако большинство существующих инструментов обеспечивает создание лишь линейных статичных документов: текстовые процессоры не поддерживают пошаговые инструкции, вкладки, сворачиваемые блоки и тесты для самопроверки. Кроме того, документы оказываются привязаны к платформе и не сохраняются как независимые переносимые файлы. В связи с этим *актуальной* задачей является разработка формата и инструментов для создания документов, сочетающих интерактивность, портативность и возможность верификации.

Объектом данной курсовой работы является процесс создания, публикации и верификации интерактивных учебно-информационных материалов как самостоятельных цифровых артефактов.

Предметом работы является совокупность архитектурных, программных и технологических решений, лежащих в основе разработки клиент-серверной платформы, а также новый формат интерактивного документа.

Целью курсовой работы является проектирование и разработка клиент-серверного приложения для создания, публикации и распространения интерактивных документов, а также формализация спецификации формата данных документов.

Для достижения поставленной цели необходимо решить следующие задачи:

- провести анализ предметной области интерактивных документов и выявить проблемы существующих решений;
- сформулировать требования к разрабатываемой системе;
- спроектировать архитектуру клиент-серверного приложения и модели данных;
- разработать и формализовать спецификацию формата интерактивного документа;
- реализовать серверную часть платформы;
- реализовать клиентскую часть платформы;
- провести тестирование разработанного программного обеспечения;
- подготовить сопроводительную документацию и инструкцию по развертыванию.

Ожидаемым результатом работы является программный продукт (клиент-серверное приложение), предоставляющий пользователям возможности по созданию, публикации, верификации и распространению интерактивных документов, а также пакет документации, включающий спецификацию формата и руководство по эксплуатации системы.

1 АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ

1.1 Обследование предметной области

Все время существования цивилизации форма представления информации во многом определяла, насколько глубоко она будет понята и усвоена. В наши дни, когда объем доступных данных огромен, главная проблема заключается уже не в их количестве, а в том, как удобно и эффективно с ними работать.

Традиционные цифровые форматы, такие как текстовые документы и статичные веб-страницы, все чаще перестают удовлетворять современным требованиям. Они делают пользователя пассивным читателем, который может лишь прокручивать текст и рассматривать изображения. При этом многие материалы (инструкции, гайды, методички, руководства, рецепты, заметки) по своей сути требуют активного взаимодействия: развернуть нужный раздел, переключиться между вариантами, пройти пошаговую инструкцию, скопировать фрагмент кода или проверить себя.

Такая потребность существует в самых разных сферах: в корпоративной среде при создании руководств и уставов, в профессиональной деятельности при работе с технической документацией, в личном использовании для заметок и самообучения, а также в образовательной сфере. Пользователи хотят получать материалы, которые одновременно удобны для чтения, позволяют быстро находить нужную информацию и дают возможность активно с ней взаимодействовать.

От современных интерактивных документов ожидается наличие следующих ключевых характеристик [2]:

- структурированность и наглядность,
- интерактивность,
- мультимедийность,
- портативность и автономность.

Документ должен быть самодостаточным, работать независимо от конкретной платформы и предоставлять возможность долгосрочного хранения и верификации.

Исследования показывают, что интерактивный контент существенно превосходит статичный по уровню вовлеченности, запоминания и общей эффективности восприятия информации [1, 2]. Тем не менее, на практике авторы материалов постоянно сталкиваются с проблемой. Инструменты, предлагающие богатую интерактивность, обычно привязывают результат к своей закрытой экосистеме. В то же время классические портативные форматы остаются статичными и не позволяют реализовать современные сценарии взаимодействия.

Таким образом, в предметной области интерактивных документов существует устойчивый запрос на формат нового типа, который органично объединит высокую интерактивность с полной автономностью, переносимостью и открытостью.

1.2 Анализ существующих решений

В настоящее время на рынке представлено достаточно большое количество решений, позволяющих работать с цифровыми материалами. Однако при ближайшем рассмотрении становится очевидно, что большинство из них решают лишь часть задач по созданию интерактивных материалов, оставляя важные пользовательские потребности неудовлетворенными.

Одни платформы делают сильный акцент на удобстве создания и редактирования контента, но при этом жестко привязывают результат к своей экосистеме. Другие предлагают действительно богатую интерактивность, однако выдают ее в виде отдельных виджетов и компонентов, которые сложно собрать в цельный, красивый и самодостаточный документ. Третьи обеспечивают высокую независимость файла, но почти полностью лишены современных интерактивных возможностей.

Рассмотрим наиболее характерных представителей этих подходов.

Платформы типа Яндекс.Практикум и Stepik представляют собой мощные образовательные среды с хорошо проработанными интерактивными элементами: аккордеонами, вкладками, пошаговыми инструкциями, встроенными тестами и проверкой заданий. Их интерфейс современный и приятный для читателя (см. рисунок 1). Однако главный недостаток таких решений – полная закрытость. Созданный материал живет только внутри платформы. Экспортировать его в независимый интерактивный документ невозможно. Автор теряет контроль над своим контентом, а читатель не может сохранить гайд или инструкцию для дальнейшего использования.

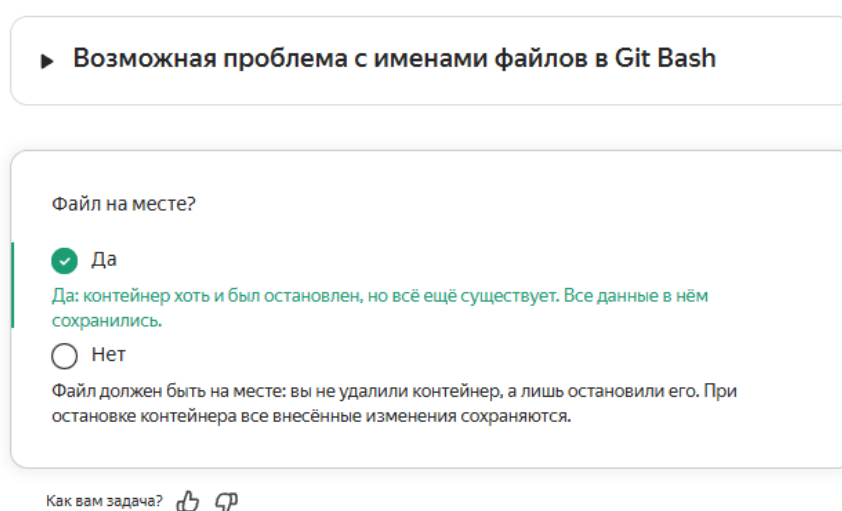


Рисунок 1 – Интерактивный интерфейс платформы Яндекс.Практикум

H5P занимает другую нишу. Это один из самых функциональных open-source инструментов для создания интерактивного контента. Платформа предлагает более 50 типов интерактивных элементов: от простых тестов и графиков до сложных симуляций (см. рисунок 2). Сильная сторона H5P – возможность встраивать созданные элементы на любые сайты. Тем не менее, собрать из них полноценный методический материал с единой структурой сложно.

Представителем третьего типа является Obsidian и подобные ему локальные базы знаний. Они обеспечивают полную независимость – все заметки хранятся в виде обычных файлов на компьютере пользователя. Это обеспечивает прекрасную переносимость и долгосрочную сохранность. Однако встроенная интерактивность в базовой версии минимальна. Для появления вкладок,

аккордеонов, тестов и других современных элементов приходится устанавливать множество плагинов (см. рисунок 3), что усложняет как создание, так и чтение таких документов другими людьми.

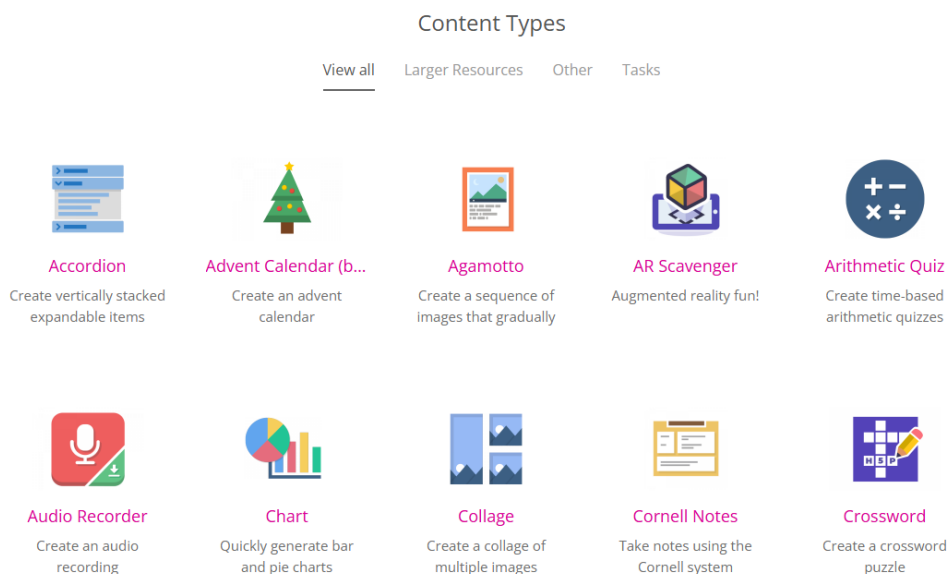


Рисунок 2 – Примеры интерактивных компонентов H5P

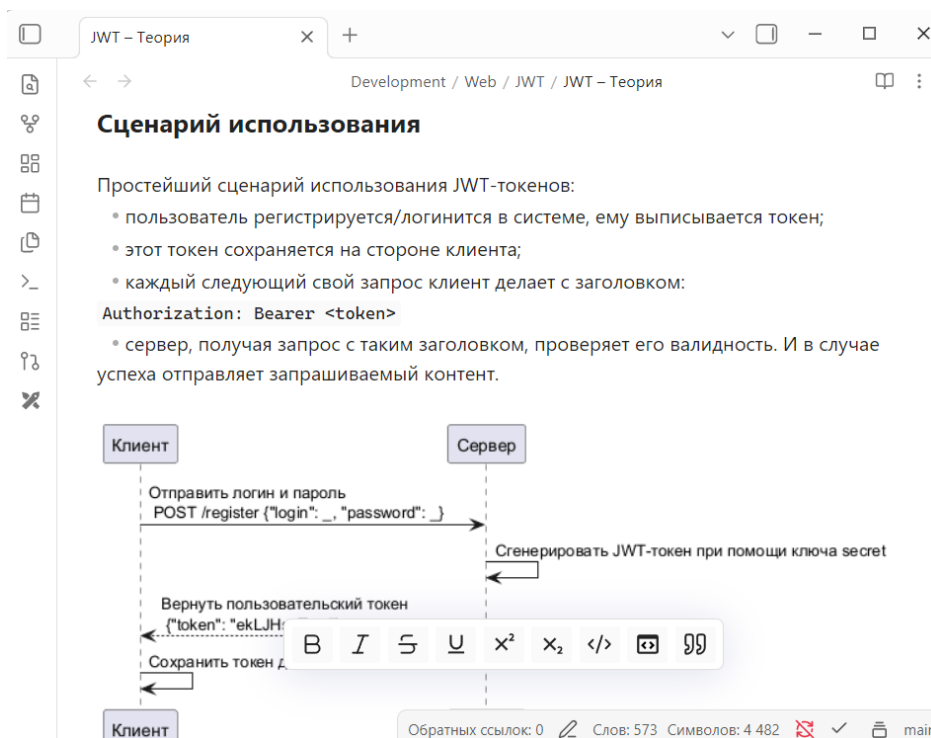


Рисунок 3 – Интерфейс Obsidian со сторонними плагинами

Видим, что существующие решения заставляют авторов постоянно выбирать между удобством и свободой: либо красивый и интерактивный

материал внутри чужой платформы, либо независимый документ, но с ограниченными возможностями взаимодействия.

Для наглядности описания характеристик существующих решений сведем их в таблицу 1.

Таблица 1 – Анализ характеристик существующих решений

Продукт	Уровень интерактивности	Переносимость и автономность	Открытость	Главные ограничения
Notion	Высокий	В рамках экосистемы	Закрытый	Сильная привязка к сервису
Obsidian	Высокий (расширяемый)	Свободная	Open-source	Другая направленность (базы знаний)
H5P	Высокий	Открытость к интеграции	Open-source	Отдельные виджеты, а не цельный документ
Moodle	Средний	Низкая	Open-source	Неповоротливость, устаревший интерфейс
Stepik / Яндекс.Практикум	Высокий	Отсутствует	Привязка к платформе	Полная зависимость от платформы
Scroll-Press	Высокий	В рамках платформы	Привязка к платформе	Научная направленность

1.3 Выявленные проблемы и ограничения

Проведенный анализ показывает, что практически все рассмотренные решения страдают от одних и тех же системных недостатков. Пользователи, создающие гайды, инструкции, методички и технические материалы, регулярно сталкиваются с ситуацией, когда готовый документ невозможно полноценно передать другому человеку или использовать вне исходной платформы. Интерактивность почти всегда достигается ценой потери независимости, а независимость – ценой потери интерактивности.

Особенно остро ощущается отсутствие открытого, универсального и самодостаточного формата интерактивного документа. На сегодняшний день нет решения, которое позволяло бы создать материал один раз и затем свободно использовать его где угодно.

Именно эти нерешенные проблемы – жесткая привязка к платформам, недостаточная переносимость, разрыв между богатой формой и цельным содержанием, а также отсутствие единого открытого стандарта – определяют высокую актуальность разработки нового интерактивного формата документа и соответствующей платформы.

Разрабатываемый формат должен устранить указанные ограничения, сочетая в себе высокую интерактивность современных инструментов с полной автономностью и переносимостью классических документов. Это позволит авторам создавать по-настоящему независимые, долговечные и удобные интерактивные материалы для самых разных сфер.

1.4 Цели и назначение разработки

На основе проведенного анализа предметной области и выявленных проблем сформулируем цели разработки платформы как поддерживающей инфраструктуры для работы с интерактивными документами нового формата. Перечень целей создания системы, требуемых значений показателей и критериев оценки достижения целей представлены в таблице 2.

Таблица 2 – Перечень целей разработки платформы

№	Цель	Требуемое значение показателей	Критерии оценки достижения цели
1	Обеспечение переносимости и самодостаточности интерактивных документов	Документ нового формата является полностью автономным	Документ открывается и полностью сохраняет интерактивность в любом совместимом ридере без подключения к сети
2	Предоставление средства просмотра интерактивных документов	Платформа предоставляет ридер с поддержкой всех типов интерактивных элементов	Все интерактивные элементы, указанные в спецификации формата, отображаются в соответствии с предписанием
3	Предоставление полнофункционального редактора документов	Редактор поддерживает редактирование всех типов элементов	Автору для создания интерактивных документов достаточно средств платформы
4	Обеспечение публикации и распространения документов	Платформа позволяет публиковать документы в общий доступ	Опубликованный документ доступен по постоянной ссылке и может быть легко передан третьим лицам
5	Обеспечение верифицируемости и целостности документов	Поддерживается механизмы проверки авторства и неизменности	Документ с электронной подписью успешно проходит верификацию

Назначением разрабатываемой платформы является предоставление авторам современного, удобного и полностью независимого инструмента для создания, публикации, распространения и долгосрочного хранения интерактивных цифровых документов.

Вид автоматизируемой деятельности – взаимодействие с интерактивными цифровыми материалами.

Система предназначена для решения следующих задач:

- создание структурированных интерактивных документов (гайды, инструкции, методички, техническая документация, заметки и др.),
- обеспечение высокой интерактивности материалов,
- хранение документов в открытом самодостаточном формате,
- обеспечение верификации целостности и авторства документов,
- предоставление возможности независимого распространения материалов без привязки к конкретной платформе,
- создание экосистемы для чтения и взаимодействия с интерактивными документами.

Объектом автоматизации системы является комплексный процесс создания, распространения и использования интерактивных цифровых документов.

1.5 Сценарии использования системы

Формализацию функциональных возможностей платформы целесообразно начинать с определения ролей пользователей и анализа сценариев их взаимодействия с системой. Такой подход позволяет четко зафиксировать требования к поведению системы с точки зрения конечных пользователей [3].

Целесообразно определить следующие основные роли пользователей:

- Администратор – лицо, осуществляющее общее управление платформой, модерацию публикаций и управление учетными записями пользователей;
- Автор – пользователь системы, создающий, редактирующий и публикующий интерактивные документы;

- Читатель – пользователь системы, который просматривает опубликованные документы и работает с ними (скачивает, добавляет в избранное, проверяет подпись);

- Гость – неавторизованный пользователь, имеющий ограниченный доступ к функциональности платформы.

Для наглядного представления возможных вариантов использования системы построим Use Case диаграмму (см. рисунок 4).

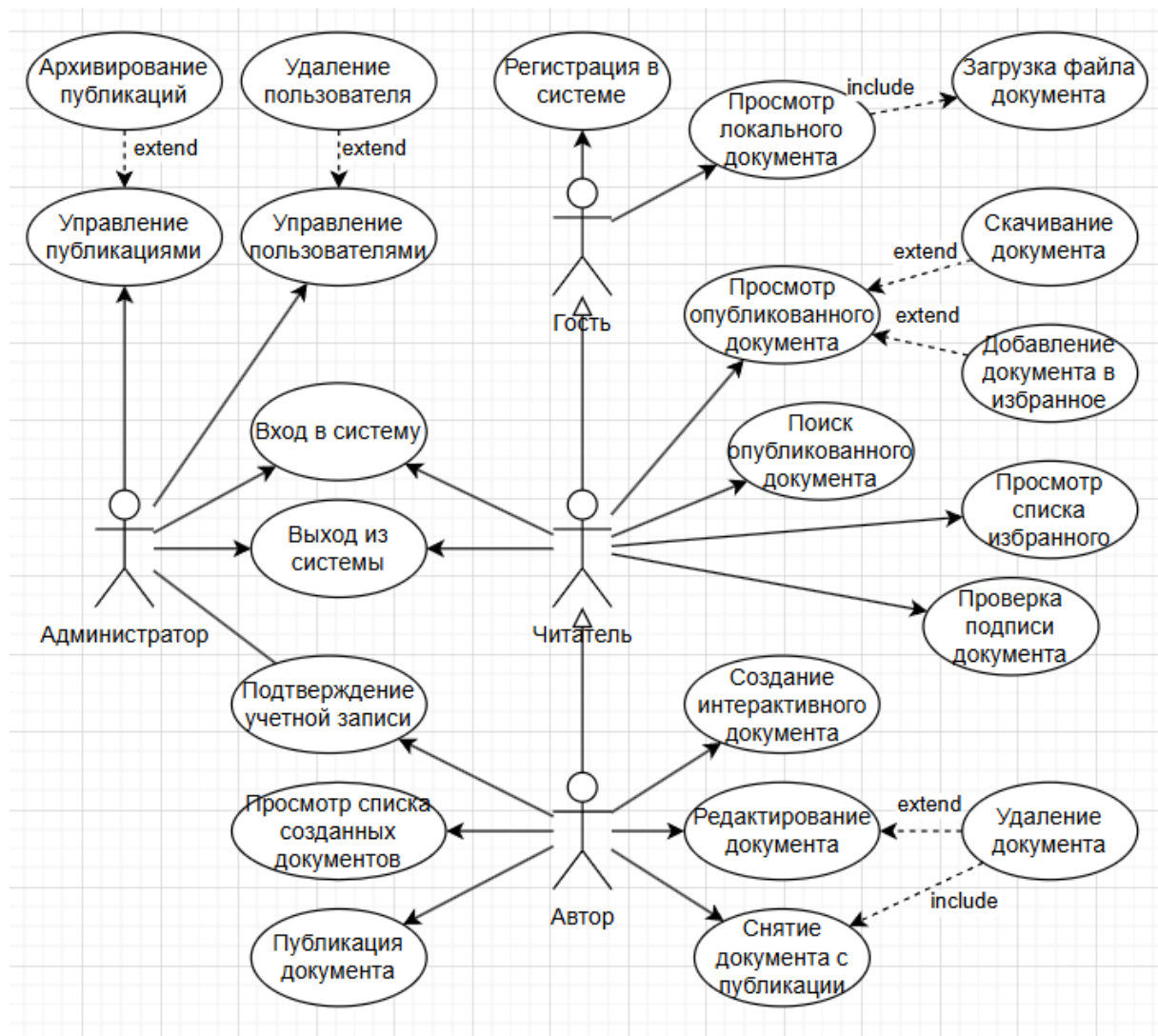


Рисунок 4 – Диаграмма вариантов использования платформы в нотации UML

Опишем каждый из представленных вариантов подробнее. Рассмотрим предусловия, которые должны быть выполнены на момент начала сценария, а также потоки действий внутри сценария. Подробный перечень сценариев использования представлен в таблице 3.

Таблица 3 – Сценарии использования системы

ID	Наименование	Актор	Предусловие	Основной поток
UC-1	Регистрация в системе	Гость	Пользователь находится на странице регистрации	1. Пользователь вводит учетные данные
				2. Система валидирует данные
				3. Система создает новую учетную запись
UC-2	Вход в систему	Любой зарегистрированный пользователь	Пользователь не аутентифицирован	1. Пользователь вводит учетные данные
				2. Система выполняет аутентификацию
				3. Пользователь перенаправляется в личный кабинет
UC-3	Создание интерактивного документа	Автор	Пользователь авторизован как Автор	1. Автор открывает редактор
				2. Создает новый документ
				3. Система сохраняет документ как черновик
UC-4	Редактирование документа	Автор	Документ существует и принадлежит Автору	1. Автор выбирает документ из списка
				2. Открывает его в редакторе
				3. Вносит изменения
				4. Система сохраняет документ
UC-5	Публикация документа	Автор	Документ находится в статусе «Черновик»	1. Автор выбирает документ
				2. Иницирует публикацию
				3. Система публикует документ и делает его общедоступным
UC-6	Просмотр опубликованного документа	Читатель	Документ опубликован	1. Пользователь открывает документ по ссылке
				2. Система рендерит документ
UC-7	Скачивание документа	Читатель	Документ опубликован	1. Пользователь открывает документ
				2. Нажимает кнопку «скачать»
				3. Система предоставляет файл документа
UC-8	Поиск опубликованных документов	Читатель	Пользователь находится на странице поиска	1. Пользователь вводит поисковый запрос
				2. Система отображает подходящие результаты

Продолжение таблицы 3

UC-9	Добавление документа в избранное	Читатель	Пользователь авторизован	1. Пользователь открывает документ
				2. Добавляет его в избранное
UC-10	Просмотр локального документа	Читатель / Гость	У пользователя есть файл интерактивного документа	1. Пользователь открывает страницу просмотра документа
				2. Загружает файл документа
				3. Система рендерит содержимое документа
UC-11	Проверка подписи документа	Читатель / Гость	Документ открыт	1. Пользователь инициирует проверку подписи
				2. Система выполняет верификацию
				3. Отображает результат проверки
UC-12	Управление публикациями	Администратор	Пользователь авторизован как Администратор	1. Администратор переходит в раздел модерации
				2. Иницирует действия над публикациями (снятие, возврат)
				3. Система выполняет действия
UC-13	Управление пользователями	Администратор	Пользователь авторизован как Администратор	1. Администратор открывает список пользователей
				2. Иницирует действия над пользователями (блокировка, подтверждение учетки)
				3. Система выполняет действия

1.6 Формирование требований к продукту

1.6.1 Функциональные требования

Проведенная формализация сценариев использования позволяет выделить ключевые функциональные возможности платформы. На основе прецедентов сформируем перечень функциональных требований, описывающих ожидаемое поведение системы при взаимодействии с пользователями.

Сгруппируем требования по функциональным блокам и зададим им идентификаторы для удобства дальнейшей ссылки и верификации [4].

Полный перечень функциональных требований к системе с указанием результатов выполнения представлен в таблице 4.

Таблица 4 – Требования к функциям, выполняемым системой

ID	Функция	Результат выполнения	Основа
FR-1	Управление учетными записями		
FR-1.1	Регистрация пользователя	Создана новая учетная запись пользователя	UC-1
FR-1.2	Аутентификация пользователя	Пользователь успешно вошел в систему	UC-2
FR-1.3	Авторизация пользователя	Пользователю назначены права в соответствии с его ролью	UC-2
FR-2	Написание документов		
FR-2.1	Создание интерактивного документа	Создан новый документ со статусом «Черновик»	UC-3
FR-2.2	Редактирование документа	Сохранено измененное состояние документа	UC-4
FR-2.3	Публикация документа	Документ опубликован и доступен по открытой ссылке	UC-5
FR-2.4	Снятие документа с публикации	Документ снят с публикации и не доступен публично	UC-5
FR-3	Взаимодействие с документами		
FR-3.1	Отображение опубликованного документа	Документ отображается с полной интерактивностью	UC-6
FR-3.2	Скачивание документа	Пользователь получил файл документа	UC-7
FR-3.3	Поиск опубликованных документов	Пользователю отображены результаты поиска	UC-8
FR-3.4	Добавление документа в избранное	Документ добавлен в пользовательский список избранного	UC-9
FR-3.5	Отображение локального документа	Подгруженный файл документа успешно открыт и отображен	UC-10
FR-4	Верификация документов		
FR-4.1	Подпись документа	При публикации документ подписан системой	UC-11
FR-4.2	Проверка подписи документа	Выполнена верификация подписи и отображен ее результат	UC-11
FR-5	Администрирование системы		
FR-5.1	Управление публикациями	Выполнены административные действия над публикациями (снятие, возвращение и т.д.)	UC-12
FR-5.2	Управление пользователями	Выполнены действия над учетными записями (подтверждение, блокировка и т.д.)	UC-13

1.6.2 Нефункциональные требования

Ожидаемые качественные характеристики системы, включая требования к показателям назначения, различным видам обеспечения, защите информации и общие технические требования, опишем в виде нефункциональных требований.

Полный перечень нефункциональных требований к системе представлен в таблице 5.

Таблица 5 – Нефункциональные требования к системе

ID	Предмет требования	Требование
NFR-1	Требования к информационному обеспечению системы	
NFR-1.1	Хранение данных	Данные пользователей и метаданные документов должны храниться в централизованной базе данных, обеспечивающей целостность и непротиворечивость
NFR-1.2	Хранение файлов	Файлы документов должны храниться в файловой системе сервера
NFR-1.3	Контрольные суммы	Для содержимого файлов должна вычисляться и храниться контрольная сумма для обеспечения целостности
NFR-1.4	Отделение публикаций	Публикации должны представлять собой отдельную от черновика документа сущность в целях автономного редактирования
NFR-2	Требования к техническому обеспечению системы	
NFR-2.1	Серверная платформа	Серверная часть системы должна корректно функционировать на операционных системах Linux на базе Debian
NFR-2.2	Контейнеризация	В целях упрощения развертывания система должна поддерживать контейнеризацию
NFR-2.3	Сетевое взаимодействие	Взаимодействие между клиентом и сервером должно осуществляться по протоколу HTTPS
NFR-2.4	Клиентская часть	Клиентская часть системы должна быть кроссплатформенной (например, веб-браузер)
NFR-3	Требования к методическому обеспечению	
NFR-3.1	Спецификация формата документа	Должна быть разработана спецификация файлового формата, включающая описание структуры документа и правила валидации
NFR-3.2	Руководство по развертыванию	Должно быть разработано руководство по развертыванию системы

Продолжение таблицы 5

NFR-4	Требования к показателям назначения	
NFR-4.1	Время аутентификации	Время выполнения операции входа в систему не должно превышать 2 секунд
NFR-4.2	Загрузка документа	Время загрузки файла документа не должно превышать 5 секунд
NFR-4.3	Верификация подписи	Время проверки подписи документа не должно превышать 3 секунд
NFR-4.4	Парсинг документа	Время извлечения содержимого файла документа не должно превышать 5 секунд
NFR-4.5	Поиск документов	Время поиска по запросу не превышает 3 секунд до первого результата
NFR-4.6	Количество пользователей	Система должна поддерживать одновременную работу не менее 50 пользователей
NFR-4.7	Объем хранимых публикаций	Максимальное количество хранимых публикаций должно быть не менее 5000
NFR-4.8	Максимальный размер документа	Система должна поддерживать загрузку документов размером не более 50 МБ
NFR-5	Требования к надежности и отказоустойчивости	
NFR-5.1	Доступность системы	Доступность системы должна составлять не менее 98% рабочего времени
NFR-5.2	Восстановление после сбоев	Время восстановления системы после сбоя не должно превышать 30 минут
NFR-5.3	Целостность данных	При сбоях не должно происходить повреждение уже сохраненных документов
NFR-5.4	Резервное копирование	Резервное копирование данных пользователей и файлов документов должно проводиться не реже одного раза в сутки
NFR-6	Требования к безопасности	
NFR-6.1	Шифрование передачи данных	Передача данных между клиентом и сервером должна осуществляться с использованием TLS (HTTPS)
NFR-6.2	Хранение паролей	Пароли пользователей должны храниться в хэшированном виде
NFR-6.3	Хранение токенов	Токены доступа не должны храниться на сервере
NFR-6.4	Подпись документов	Публикуемые документы должны подписываться на сервере
NFR-6.5	Защита от уязвимостей	Система должна быть защищена от типовых веб-уязвимостей (XSS, CSRF, SQL-injection)
NFR-6.6	Хранение секретов	Секретные ключи не должны храниться в коде или репозитории
NFR-7	Требования к защите информации от несанкционированного доступа	
NFR-7.1	Разграничение доступа	Доступ к функциям системы должен определяться ролями пользователей
NFR-7.2	Аутентификация	Доступ к ресурсам API осуществляется только при наличии валидного токена
NFR-7.3	Контроль сессий	Refresh-токены должны иметь ограниченное время жизни и могут быть отозваны

1.6.3 Ограничения системы

В процессе анализа предметной области были выявлены границы, в пределах которых разрабатываемая система будет решать поставленные задачи. Данные ограничения определяются спецификой целевых сценариев использования, практическими потребностями пользователей и сложившейся практикой работы с интерактивными документами. Система не претендует на универсальность, и осознанное сужение функциональности позволяет сфокусироваться на ключевой проблеме – создании и распространении переносимых интерактивных документов.

Перечень ограничений системы представлен в таблице 6.

Таблица 6 – Ограничения разрабатываемой системы

ID	Ограничение	Обоснование
CON-1	Отсутствие совместного редактирования в реальном времени	Целевой сценарий системы – создание готовых авторских материалов, а не коллективная работа над одним документом
CON-2	Отсутствие поддержки выполняемого кода внутри документа	Основная цель формата и платформы – безопасное распространение материалов для чтения и взаимодействия через предусмотренные интерфейсы. Наличие исполняемого кода создает потенциальные риски для читателя и требует дополнительных мер безопасности, что противоречит принципу переносимости
CON-3	Ограничение максимального размера документа	Типовые целевые материалы редко превышают десятки мегабайт даже с учетом мультимедиа
CON-4	Отсутствие автоматической синхронизации между устройствами читателя	Документ задуман как самостоятельный переносимый артефакт, который пользователь сохраняет там, где считает нужным, и передает по своему усмотрению. Привязка к конкретному аккаунту или автоматическая синхронизация между устройствами не является обязательной для целевых сценариев чтения и использования
CON-5	Одноязычный интерфейс	На начальном этапе разработки целевая аудитория ограничена русскоязычными пользователями
CON-6	Отсутствие встроенных средств для коммуникации между пользователями	Платформа ориентирована на работу с документами как с конечным продуктом. Коммуникация между авторами и читателями (обсуждение, вопросы, обратная связь) не является функцией, решаемой в рамках данной работы

1.6.4 Требования к формату интерактивного документа

Формат интерактивного документа является ключевым предметом разрабатываемой платформы. Именно он обеспечивает переносимость, автономность и интерактивность материалов. Сформулируем основные требования к нему:

1. DFR-1. Платформонезависимость. Документ не должен быть жестко привязан к HTML, веб-движку или конкретной операционной системе. Его структура должна позволять реализовывать ридеры для различных платформ: веб, мобильные устройства, десктопные приложения.
2. DFR-2. Самодостаточность. Документ должен включать все необходимые ресурсы (изображения, видео) внутри себя либо ссылаться на них явно. Открытие документа не должно требовать доступа к внешним серверам для получения базового содержимого.
3. DFR-3. Отсутствие состояния (stateless). Документ не должен хранить информацию о действиях читателя. Все временное состояние управляется исключительно ридером в процессе одного сеанса чтения.
4. DFR-4. Детерминированность отображения. При одном и том же файле документа совместимый ридер должен обеспечить идентичное начальное отображение при любом количестве чтений.
5. DFR-5. Поддержка интерактивных элементов. Формат должен предоставлять средства для описания структур, обеспечивающих активное взаимодействие читателя с содержимым: вкладки, пошаговые инструкции, сворачиваемые блоки, тесты для самопроверки, галереи изображений.
6. DFR-6. Возможность верификации. Формат должен поддерживать механизмы проверки целостности документа и подлинности авторства.
7. DFR-7. Расширяемость и обратная совместимость. Спецификация формата должна допускать добавление новых типов элементов в будущих версиях без поломки существующих ридеров. Ридеры должны уметь адаптироваться к неизвестным элементам.

1.7 Итоги анализа предметной области

В результате проведенного анализа предметной области были получены следующие результаты.

Приобретено понимание процессов и проблем. Выявлено противоречие между потребностью в интерактивных материалах и существующими решениями: интерактивность достигается ценой привязки к платформе, а переносимость – ценой потери интерактивности. Установлено, что на рынке отсутствует открытый самодостаточный формат, сочетающий обе характеристики.

Определение сущностей и ролей. Выделены ключевые роли пользователей (Гость, Читатель, Автор, Администратор) и основные сущности предметной области (пользователь, документ-черновик, публикация). Сформулированы сценарии использования системы, охватывающие полный жизненный цикл документа – от создания до верификации.

Сформированы требования. Разработаны функциональные требования (управление учетными записями, создание и публикация документов, взаимодействие с документами, верификация, администрирование), нефункциональные требования (информационное, техническое, методическое обеспечение, показатели назначения, надежность, безопасность), а также ограничения системы и требования к формату интерактивного документа.

Совокупность полученных результатов создает информационную основу для проектирования разрабатываемой системы.

2 ПРОЕКТИРОВАНИЕ СИСТЕМЫ

2.1 Концепция продукта

На основе проведенного в первой главе анализа предметной области и сформированных требований обобщим представление о разрабатываемом продукте в виде его концепции.

2.1.1 Общая характеристика продукта

В целях идентификации продукта и создания бренда было принято решение о наименовании разрабатываемой платформы как «Doseum».

Doseum представляет собой клиент-серверную платформу с открытым исходным кодом, предназначенную для создания, публикации, распространения и верификации интерактивных документов. Ключевая особенность продукта – собственный самодостаточный файловый формат .doseo (от лат. doseo – учу, преподаю), сочетающий высокую интерактивность (вкладки, пошаговые инструкции, аккордеоны, тесты) с полной переносимостью и независимостью от конкретной платформы.

Продукт обеспечивает:

- создание и редактирование интерактивных документов в визуальном редакторе,
- публикацию документов в общедоступный каталог,
- просмотр опубликованных и локальных документов в специализированном ридере,
- скачивание документов в формате .doseo для дальнейшего использования,
- верификацию целостности и подлинности документов через механизм подписи,
- поиск опубликованных документов по каталогу.

2.1.2 Особенности и принципы реализации

Разработка Doseum строится на следующих ключевых принципах:

1. **Формат как спецификация.** Основной ценностью продукта является открытая спецификация формата .doseo. Любой разработчик может реализовать совместимый ридер или редактор, не привязываясь к конкретной реализации платформы.
2. **Платформонезависимость документа.** Документ .doseo не привязан к HTML, веб-движку или операционной системе. Его структура позволяет создавать ридеры для любых сред: веб, мобильных устройств, десктопных приложений.
3. **Отсутствие состояния и самодостаточность.** Документ не хранит состояние читателя и включает все необходимые ресурсы (изображения, видео) внутри себя. Открытие документа не требует доступа к внешним серверам для получения базового содержимого.
4. **Подпись как доказательство происхождения.** Публикуемые документы подписываются на сервере. Читатель может в любой момент проверить подпись и убедиться, что документ не был изменен после публикации и действительно создан автором на данной платформе.
5. **Открытость и расширяемость.** Исходный код платформы публикуется под открытой лицензией. Спецификация формата предусматривает обратную совместимость при добавлении новых типов блоков.

2.1.3 Целевая аудитория

Целевой аудиторией Doseum являются специалисты, создающие или использующие структурированные интерактивные материалы. В первую очередь это преподаватели и методисты – школьные учителя, преподаватели вузов и колледжей, разработчики дидактических материалов, репетиторы и авторы онлайн-курсов. Вторую крупную категорию составляют технические специалисты: технические писатели, разработчики ПО и инженеры, создающие руководства, инструкции и архитектурную документацию. Третья

категория – корпоративные специалисты, включая HR-менеджеров и сотрудников отделов внутреннего обучения, которые разрабатывают онбординг-гайды и регламентную документацию. Кроме того, платформа ориентирована на самостоятельных авторов – блогеров, экспертов и создателей инструкций для широкой аудитории. В случае расширения проекта Doseum также может стать полезным инструментом для профессиональных сообществ и некоммерческих проектов, предъявляющих высокие требования к переносимости и сохранности материалов.

2.2 Функциональная модель системы

Для формализованного описания функциональности разрабатываемой системы построим ее функциональную модель в нотации IDEF0.

Целью моделирования является формализация процессов создания, публикации, распространения и отображения интерактивных документов. Точка зрения модели – платформа Doseum как совокупность реализуемых функций.

На контекстной диаграмме (см. рисунок 5) система представлена функциональным блоком А0 «Обеспечение комплексного взаимодействия с интерактивными документами».

На диаграмме декомпозиции (см. рисунок 6) эта функция детализирована на пять связанных подфункций:

- А1 «Управление учетными записями пользователей»,
- А2 «Создание и редактирование документов»,
- А3 «Публикация документа в каталог платформы»,
- А4 «Отображение интерактивных документов»,
- А5 «Поиск опубликованных документов в каталоге».

Функции А2, А3, А4 образуют последовательный поток обработки документа: создание, публикация, отображение. Функция А5 обеспечивает поиск по опубликованным документам, работая на основе данных, сформированных функцией А3. Дальнейшая декомпозиция блоков в рамках данной работы не производится.

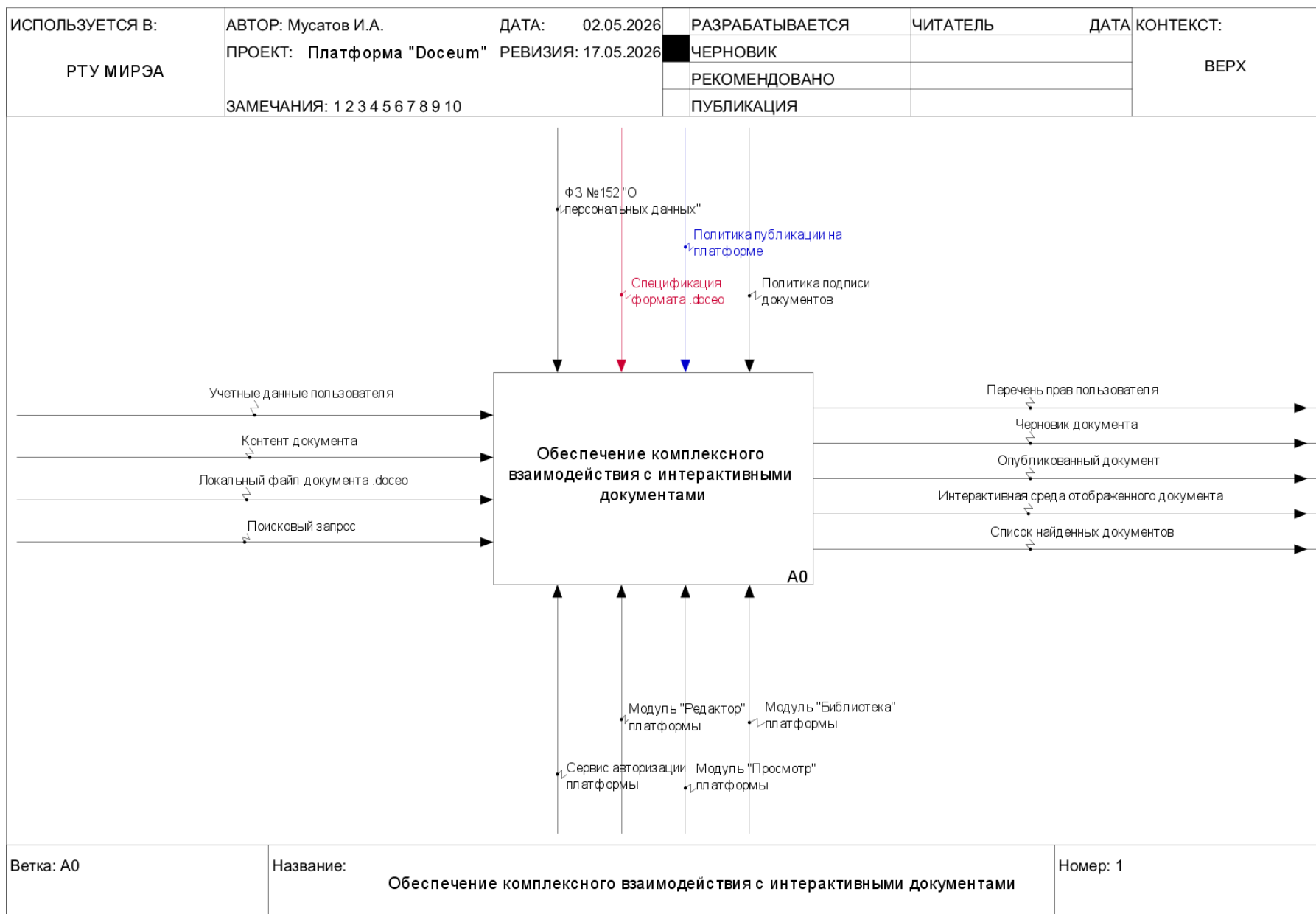


Рисунок 5 – Контекстная диаграмма функциональной модели системы «Doseum» в нотации IDEF0

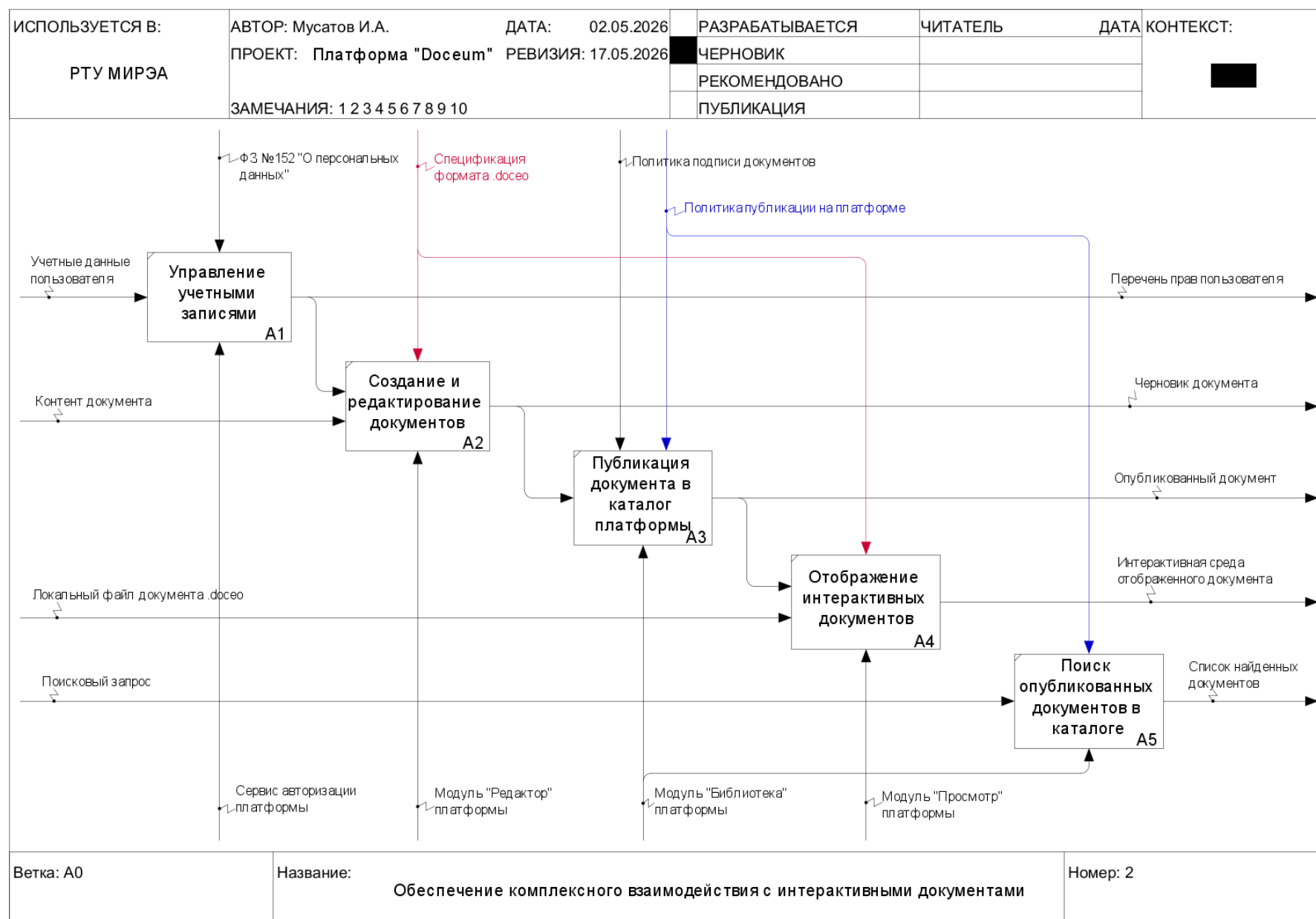


Рисунок 6 – Диаграмма декомпозиции A0 функциональной модели системы «Doseum» в нотации IDEF0

2.3 Обоснование архитектурных решений

Выбор архитектуры программной системы является одним из наиболее важных этапов проектирования, поскольку он напрямую влияет на масштабируемость, сопровождаемость, безопасность и удобство использования продукта. В данном разделе обосновывается выбор основных архитектурных решений для платформы Docsum.

2.3.1 Обоснование выбора вида архитектуры

В качестве архитектуры платформы была выбрана клиент-серверная архитектура. Такое решение обусловлено следующими факторами:

1. Создание полнофункциональной экосистемы интерактивных документов. Клиент-серверная модель позволяет обеспечить единую среду, в которой авторы могут создавать и публиковать документы, а читатели находить, просматривать и скачивать их.
2. Централизованное управление публикациями и доступом. Платформа должна обеспечивать публикацию документов в общий каталог, модерацию контента и управление правами доступа.
3. Обеспечение верифицируемости. Электронная подпись документов и проверка их целостности должны выполняться на доверенной серверной стороне с использованием секретного ключа платформы.
4. Удобство обновлений. При клиент-серверной модели обновление функциональности ридера и средства просмотра происходит централизованно, делая актуальную версию общедоступной.
5. Масштабируемость и расширяемость. Архитектура позволяет в будущем расширять систему, добавляя новые сервисы и поддерживая большое количество пользователей и документов.

Альтернативные варианты – централизованная (локальная), «файл-сервер» и распределенная архитектуры – были отклонены как не соответствующие специфике платформы и требованиям к ней.

2.3.2 Обоснование выбора типа клиентской части

В качестве клиентской части платформы выбран веб-интерфейс, реализованный по архитектуре Single Page Application (SPA). Такое решение является наиболее обоснованным для достижения целей проекта.

Веб-подход обеспечивает максимальную кроссплатформенность и доступность: пользователь может работать с платформой с любого устройства (компьютер, планшет, смартфон), имеющего современный браузер, без необходимости установки дополнительного программного обеспечения. Это особенно важно для проекта, ориентированного на широкое распространение интерактивных документов. Порог входа для читателей и авторов должен быть минимальным.

Кроме того, веб-реализация значительно упрощает распространение и обновление платформы. В отличие от нативных приложений, изменения в редакторе, ридере или новых типах интерактивных элементов сразу становятся доступны всем пользователям без необходимости обновления приложений через магазины или установщики. Это критично на этапе активной разработки формата .docseo, когда функциональность будет регулярно расширяться.

Нативные desktop и мобильные приложения обладают важным преимуществом: возможностью полноценной работы в оффлайн-режиме без подключения к интернету. Однако на текущем этапе разработки их реализация существенно усложнила бы процесс разработки и поддержки, потребовав создания и сопровождения нескольких отдельных кодовых баз. Ввиду этого нативные приложения рассматриваются лишь как перспективное направление расширения экосистемы Docseum.

2.3.3 Обоснование архитектурного стиля серверной части

Для серверной части платформы выбран модульный монолит (Modular Monolith / Modulith) с применением принципов Clean Architecture.

На текущем этапе разработки объем бизнес-логики относительно небольшой. Основная работа сервера сводится к обработке документов,

управлению пользователями, публикациям и верификации подписи. В этих условиях переход на микросервисную архитектуру был бы избыточным и непродуктивным. Микросервисы значительно усложнили бы разработку, тестирование и отладку, особенно учитывая, что над проектом работает один разработчик.

Модульный монолит позволяет сохранить все преимущества монолитной архитектуры (простота разработки, единая кодовая база, отсутствие сетевого взаимодействия между компонентами), при этом обеспечивая хорошую структурированность и масштабируемость в будущем [5]. Каждый крупный функциональный блок (авторизация, работа с документами, публикации, профиль) выделяется в отдельный модуль с четкими границами ответственности.

Применение Clean Architecture внутри модулей позволяет добиться высокой тестируемости, независимости бизнес-логики от внешних фреймворков и базы данных, а также упрощает поддержку и развитие системы в долгосрочной перспективе [6].

Микросервисный подход планируется рассмотреть только на следующих этапах развития проекта, когда нагрузка и сложность функциональности существенно возрастут и появится необходимость в независимом масштабировании отдельных компонентов.

Таким образом, выбранный архитектурный стиль представляет собой оптимальный баланс между простотой разработки на текущем этапе и возможностью дальнейшего эволюционного развития системы.

2.4 Проектирование архитектуры системы

Для описания архитектуры платформы Doseum построим диаграммы различных уровней абстракции. Такой подход позволяет последовательно перейти от общего представления системы к деталям реализации ее основных компонентов.

2.4.1 Схема контейнеров системы

Для представления верхнего уровня абстракции построим диаграмму контейнеров в нотации C4 (см. рисунок 7). Данная диаграмма отражает основные исполняемые единицы (контейнеры) системы и взаимодействия между ними. Ключевыми контейнерами платформы являются: сервер приложений (RESTful API), веб (и обратный прокси) сервер, веб-приложение (SPA), WebAssembly-парсер, база данных для кэша, основная реляционная база данных, а также файловое хранилище.

Диаграмма показывает, что основное взаимодействие между клиентом и сервером осуществляется через RESTful API, а операции по обработке документов частично вынесены на клиентскую сторону с помощью WebAssembly.

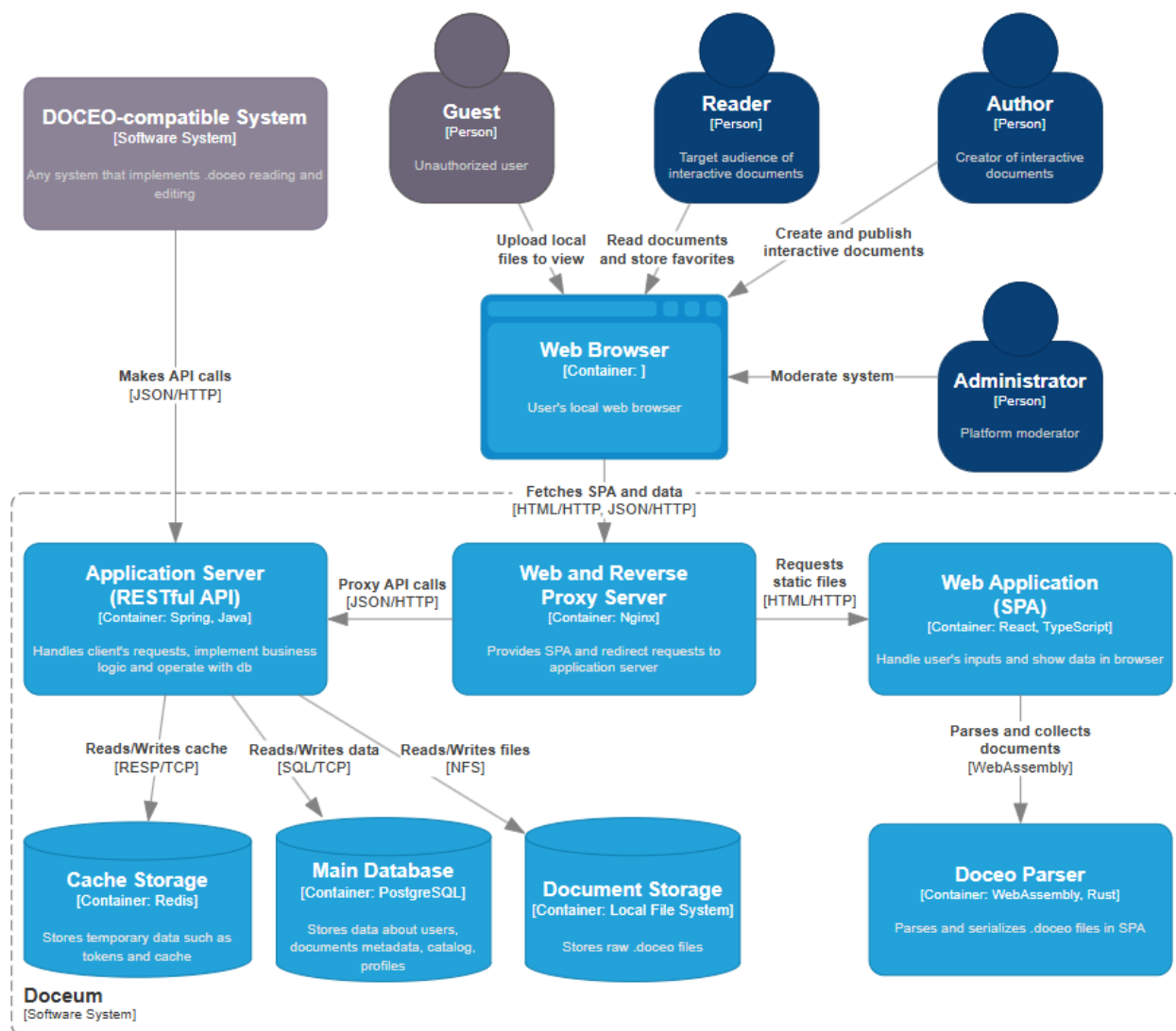


Рисунок 7 – Диаграмма контейнеров системы Doseum в нотации C4

2.4.2 Структура компонентов серверной части

Серверная часть платформы Досеит реализована по архитектуре модульного монолита. Диаграмма компонентов (см. рисунок 8) отражает основные модули бэкэнд-приложения и их внутреннюю организацию.

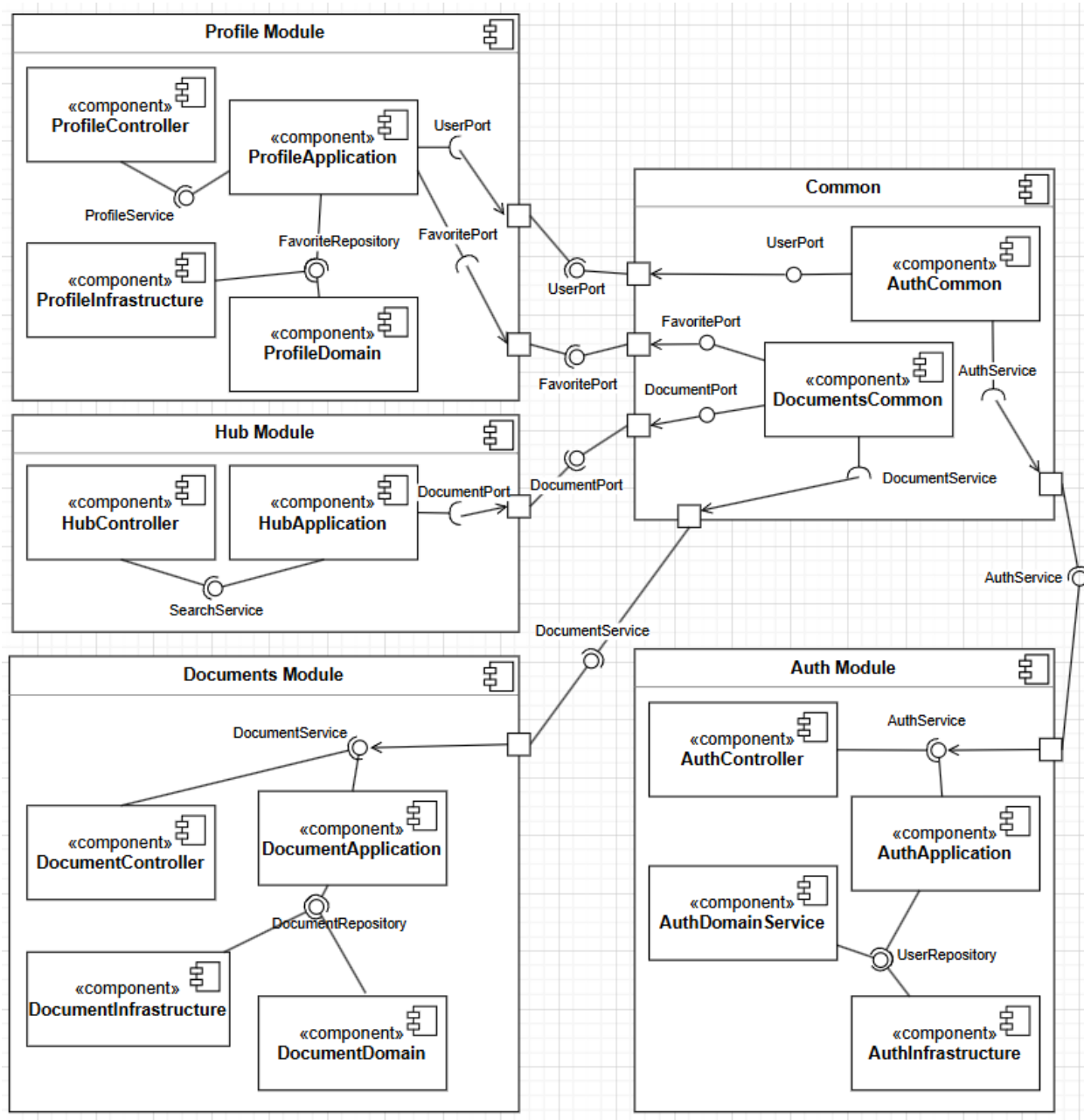


Рисунок 8 – Диаграмма компонентов серверной части в нотации UML

Серверная часть системы разделена на четыре основных функциональных модуля:

- Auth Module отвечает за аутентификацию, авторизацию и управление пользователями;

- Documents Module – центральный модуль системы, обеспечивающий создание, редактирование, парсинг, генерацию и подпись документов формата .docseo;
- Hub Module отвечает за публикацию документов, поиск и работу с публичным каталогом;
- Profile Module нацелен на управление профилем автора, избранным и списком документов.

Каждый модуль построен по принципам чистой архитектуры (Clean Architecture) и содержит следующие основные компоненты [6]:

- API Layer – REST-контроллеры, принимающие HTTP-запросы;
- Application Layer – application services и use cases, реализующие бизнес-логику;
- Domain Layer – доменные модели, value objects и доменные сервисы;
- Infrastructure Layer – реализация портов (репозитории, работа с файловой системой, внешние сервисы).

Такое разделение обеспечивает низкую связанность между модулями, высокую тестируемость и удобство дальнейшего сопровождения и развития системы.

2.4.3 Архитектура развертывания системы

Диаграмма развертывания (см. рисунок 9) иллюстрирует физическое размещение компонентов платформы Docseum и выбранную инфраструктуру.

Для обеспечения простоты развертывания, воспроизводимости окружения и удобства разработки выбраны технологии Docker и Docker Compose. Все части системы упакованы в контейнеры, что позволяет запускать платформу на любой машине с минимальными зависимостями.

Основные компоненты развертывания включают:

- Nginx – веб-сервер, обслуживающий статические файлы фронтенда и выступающий в роли reverse proxy;
- Backend Application – основной серверный контейнер (Java/Spring Boot);
- PostgreSQL – контейнер с реляционной базой данных;
- Redis – контейнер для кэширования и хранения refresh-токенов;
- File Storage – Docker том для хранения файлов формата .docoo.

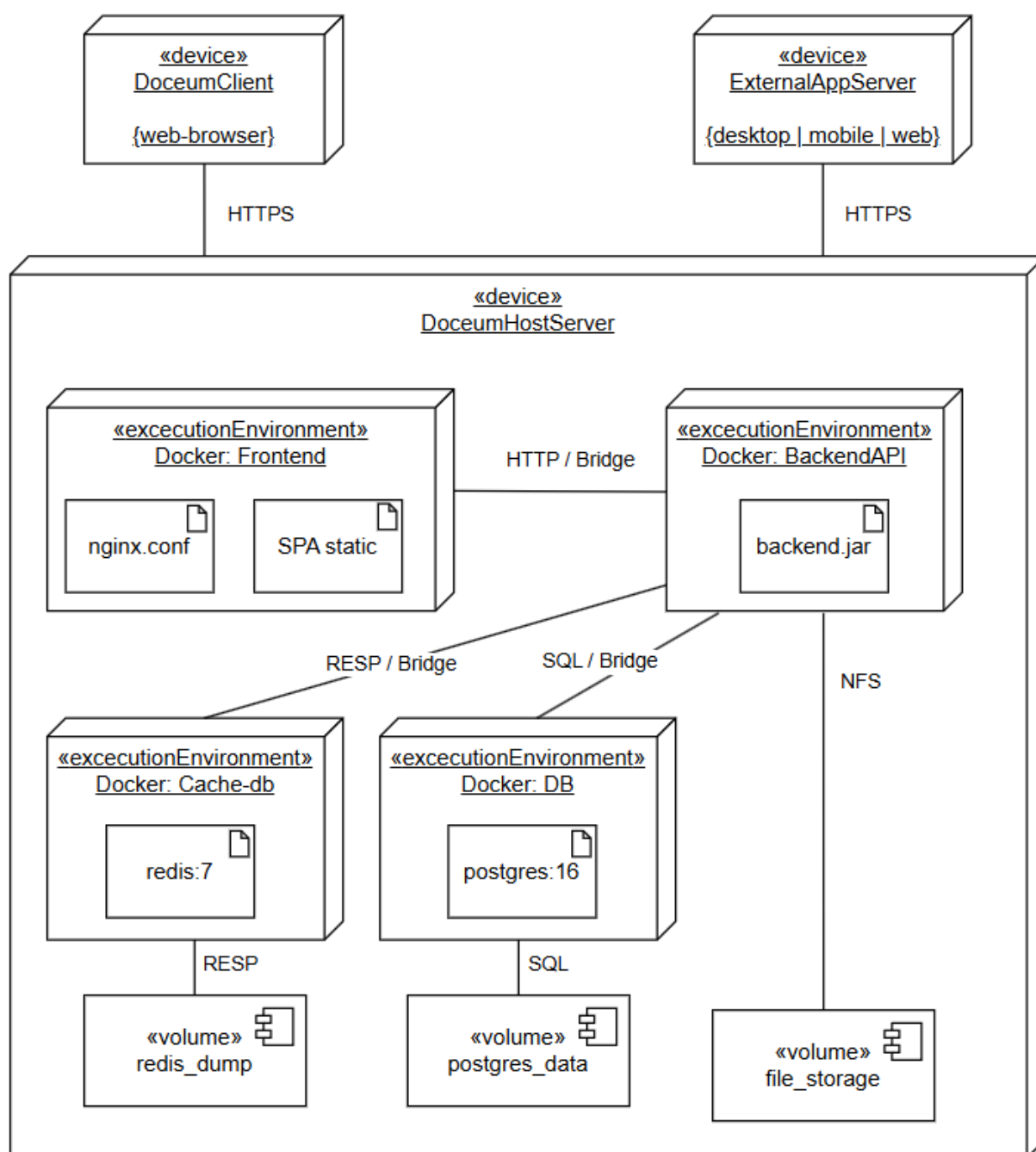


Рисунок 9 – Диаграмма развертывания платформы Досеум в нотации UML

2.4.4 Сценарии взаимодействия компонентов системы

Для детального описания взаимодействия компонентов системы была построена диаграмма последовательности для процесса «Редактирование документа» (см. рисунок 10).

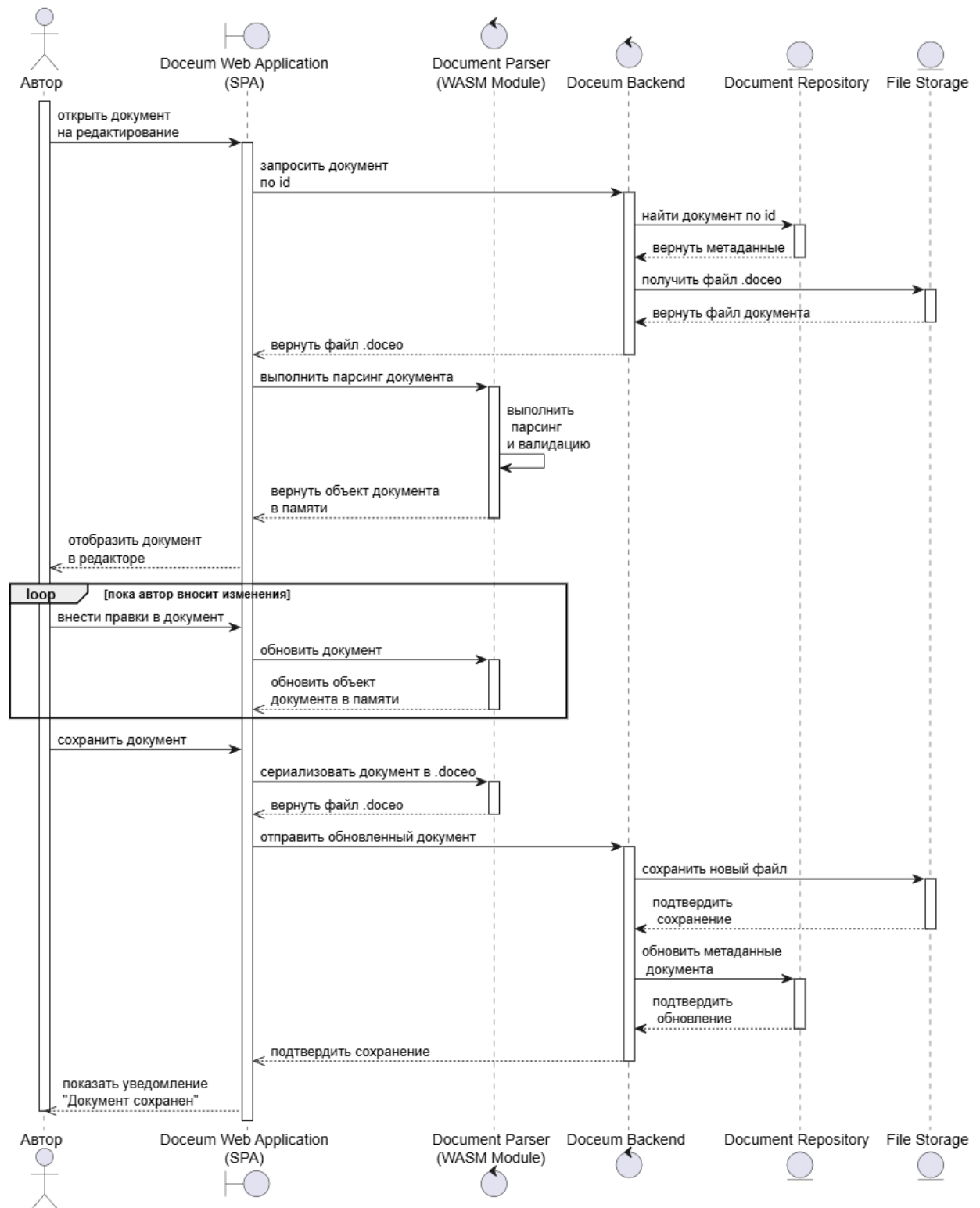


Рисунок 10 – Диаграмма последовательности в нотации UML

Данный процесс наиболее полно отражает сложное взаимодействие между клиентской и серверной частями платформы, а также вовлеченность WASM-модуля.

На диаграмме представлен цикл редактирования документа. Процесс начинается с запроса файла Автором. Doseum Web Application (SPA) получает документ с сервера и передает его в Document Renderer (WASM Module) для парсинга и построения объектной модели.

После внесения правок Автор сохраняет документ. WASM-модуль выполняет сериализацию объекта обратно в формат .doseo. Полученный файл вместе с метаданными отправляется на Doseum Backend. Сервер сохраняет файл в File Storage и обновляет метаданные в Document Repository.

Данный сценарий наглядно демонстрирует распределение ответственности между компонентами: тяжелые операции парсинга и сериализации выполняются на стороне клиента с помощью WebAssembly, что снижает нагрузку на сервер и повышает отзывчивость редактора, при этом сервер остается единственным источником истины для хранения и обеспечения целостности документов.

2.5 Проектирование формата интерактивного документа

Центральным элементом платформы Doseum является специально разработанный формат файлов .doseo. Именно он обеспечивает главную ценность проекта: сочетание высокой интерактивности с полной переносимостью и автономностью документов.

Формат .doseo представляет собой самодостаточный (self-contained) ZIP-архив, не зависящий от внешних сервисов и платформ. Он построен на следующих ключевых принципах:

1. Автономность — документ содержит все необходимые ресурсы внутри себя или явно указывает на них.
2. Отсутствие состояния (stateless) — документ не хранит текущее состояние читателя.

3. Детерминированность отображения – один и тот же файл должен одинаково отображаться в любом совместимом ридере.

4. Версионирование и обратная совместимость – формат позволяет развивать спецификацию без нарушения работы существующих ридеров.

Полная спецификация формата .doceo версии 1.0 доступна по публичной ссылке: <https://honsage.github.io/Doceum/>.

Формат использует четыре основных типа узлов:

1. Семантические контейнеры – организуют структуру документа и определяют поведение при отображении (accordion, tabs, stepper, gallery, callout и др.).

2. Семантические структуры – имеют фиксированную схему полей (heading, paragraph, table, quiz, figure и др.).

3. Носители контента – терминальные элементы, содержащие конкретный тип данных (image, video, code, formula, canvas, diagram).

4. Встроенные узлы (Inline Nodes) – формируют модель rich text внутри текстовых блоков (span, inline_code, link, spoiler, copy_snippet и др.).

Такая многоуровневая модель позволяет создавать сложные, хорошо структурированные документы с богатой интерактивностью, сохраняя при этом строгость и предсказуемость отображения.

Ключевые типы узлов формата приведены в таблице 7.

Таблица 7 – Основные типы узлов формата .doceo

Тип узла	Категория	Основное назначение
container	Семантический контейнер	Нейтральная группировка
accordion, tabs, stepper	Семантический контейнер	Интерактивная структура
heading, paragraph	Семантическая структура	Текстовое содержание
list, quote, callout	Семантическая структура	Специализированные блоки
image, video, code	Носитель контента	Медиа и код
quiz, table	Семантическая структура	Интерактивные и табличные элементы
span, inline_code, link	Встроенный узел (rich text)	Форматированный текст

Полный справочник всех поддерживаемых узлов и правил валидации также приведен в официальной спецификации формата.

2.6 Проектирование баз и хранилищ данных

Реляционная схема базы данных включает следующие таблицы: User (учетные записи, роли, персональные данные), Document (метаданные документов, статус, путь к файлу, контрольная сумма), Publication (привязка к документу, подпись, дата публикации), Favorites (избранное пользователей), ViewHistory (история просмотров публикаций).

Логическая модель базы данных представлена в виде ER-диаграммы (см. рисунок 11).

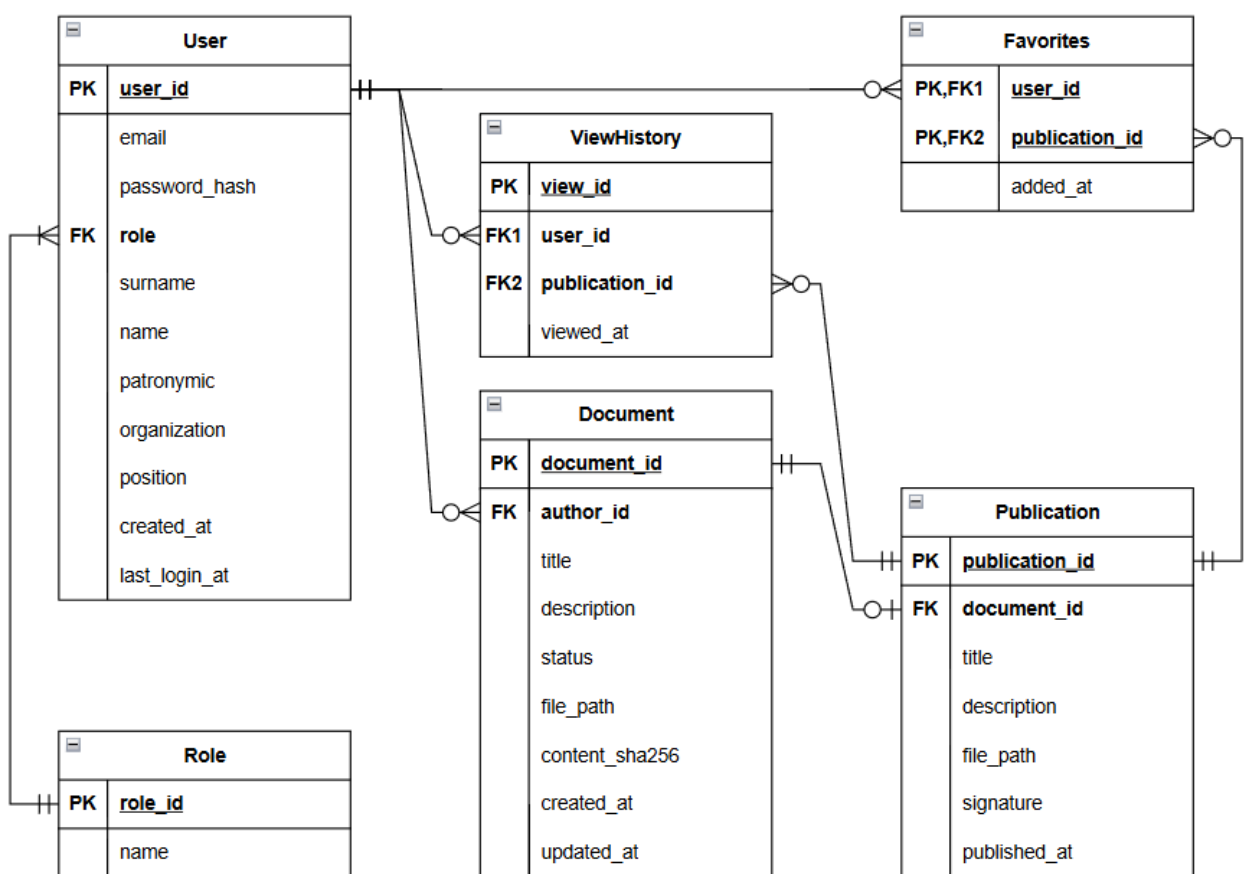


Рисунок 11 – Логическая модель базы данных системы в нотации ERD

Опишем также компоненты хранения, выходящие за рамки реляционной модели.

Файлы в формате .docx хранятся в файловой системе сервера. Файловая система организована следующим образом: есть корневая директория, в которой расположены две папки: drafts/ и publications/. Внутри папки drafts/ располагаются папки пользователей-авторов, именованные по UUID

идентификаторам авторов. Внутри папки каждого автора хранятся файлы .docseo, именованные UUID соответствующих документов. Внутри папки publications/ хранятся файлы опубликованных документов .docseo, также именованные при помощи UUID соответствующих документов.

Refresh-токены для JWT-аутентификации хранятся в нереляционной базе данных вида «ключ-значение», что обусловлено необходимостью быстрого доступа и автоматического истечения срока жизни. Структура ключей:

- refresh:{token_hash} – хранит user_id, TTL 30 дней;
- user_tokens:{user_id} – множество активных токенов пользователя, TTL 30 дней.

Access-токены реализованы как самодостаточные JWT и на сервере не хранятся, что снижает нагрузку на Redis.

2.7 Проектирование программного интерфейса

Взаимодействие клиентской части с сервером осуществляется через RESTful API, спроектированный по принципам ресурсно-ориентированной архитектуры.

Аутентификация реализована с помощью JWT (JSON Web Token). При успешном входе сервер возвращает access-токен (время жизни 15 минут) и refresh-токен (30 дней). Access-токен содержит идентификатор пользователя, роль и время истечения и используется для доступа к защищенным эндпоинтам. При его истечении клиент отправляет refresh-токен на /api/auth/refresh для получения новой пары токенов.

Ролевая модель разграничивает права доступа: READER может только просматривать и скачивать публикации; AUTHOR дополнительно создает, редактирует и публикует документы; ADMIN управляет пользователями и модерацией публикаций.

API сгруппирован по функциональным модулям, выделенным в архитектуре серверной части.

Модуль аутентификации (Auth) обеспечивает регистрацию, вход, обновление токенов и выход из системы. Эндпоинты этой группы доступны всем неавторизованным пользователям.

Модуль документов (Documents) предоставляет операции для работы с документами: создание, редактирование черновиков, публикация, снятие с публикации, просмотр, получение метаданных и верификация. Доступ к эндпоинтам редактирования и публикации ограничен для авторизованных авторов.

Модуль каталога (Hub) обеспечивает поиск и навигацию по опубликованным документам. Эндпоинты доступны всем пользователям.

Модуль профиля (Profile) управляет личными данными профилей пользователей: списком черновиков, опубликованных документов (для авторов) и избранным. Доступ только для авторизованных пользователей.

Список эндпоинтов с указанием методов приведен в таблице 8.

Таблица 8 – Перечень основных эндпоинтов API платформы Doseum

Модуль	Метод	Эндпоинт
Auth	POST	/api/auth/register
Auth	POST	/api/auth/login
Auth	POST	/api/auth/refresh
Auth	POST	/api/auth/logout
Documents	POST	/api/documents
Documents	PUT	/api/documents/{id}/draft
Documents	GET	/api/documents/{id}/draft
Documents	DELETE	/api/documents/{id}/draft
Documents	POST	/api/documents/{id}/publish
Documents	DELETE	/api/documents/{id}/publish
Documents	GET	/api/documents/{id}/view
Documents	GET	/api/documents/{id}/metadata
Documents	POST	/api/documents/verify
Hub	GET	/api/hub/recent
Hub	GET	/api/hub/documents
Hub	GET	/api/hub/documents/{id}
Profile	GET	/api/profile/favorites
Profile	POST	/api/profile/favorites/{publicationId}
Profile	DELETE	/api/profile/favorites/{publicationId}
Profile	GET	/api/profile/documents/drafts
Profile	GET	/api/profile/documents/published

2.8 Проектирование пользовательского интерфейса

Первоочередной задачей дизайна платформы стало создание спокойной, аккуратной и минималистичной цветовой схемы, которая не отвлекает от основного контента – интерактивных документов. Для платформы была выбрана нейтрально-теплая палитра с мягким мятным цветом.

Выбор цветовой гаммы основывался на следующих принципах: снижение когнитивной нагрузки и усталости глаз при длительной работе с документами (теплые нейтральные тона создают комфортный фон для продолжительного чтения и редактирования); баланс между спокойствием и функциональной выразительностью (акцентный мятный цвет объединяет свойства синего и зеленого, что имеет значение для образовательных материалов); ориентация на современные практики инструментов (аналогичная философия дизайна используется в Notion, Craft, Bear); а также учет исследований, подтверждающих, что мягкие нейтральные тона и приглушенные акценты снижают зрительную утомляемость [7].

В результате была утверждена палитра «Soft Clarity» (см. рис. 12).



Рисунок 12 – Палитра цветов дизайна пользовательского интерфейса Doseum

Для развития бренда при помощи инструмента создания векторной графики Inkscape был нарисован логотип платформы. Он представляет собой стилизованную под свитки письменную букву «D», делающую отсылку к названию и концепции платформы (см. рисунок 13).



Рисунок 13 – Логотип платформы Doseum

Дизайн пользовательского интерфейса платформы разработан при помощи инструмента Figma. Пример дизайна страницы платформы представлен на рисунке 14.

Рисунок 14 – Дизайн интерфейса страницы авторизации

Платформа реализована как Single Page Application (SPA) с клиентской маршрутизацией, что обеспечивает плавные переходы между страницами без перезагрузки. Маршрутизация построена на основе роутинга и включает следующие основные маршруты: страница входа (/login), регистрации (/register), главная страница с каталогом публикаций (/), страница просмотра документа

(/document/:id), страница загрузки локального документа (/viewer), страница редактирования документа (/editor/:id), личный кабинет автора (/profile), страница поиска (/search), а также страница 404 для несуществующих маршрутов.

Интерфейс платформы строится на компонентном подходе: ключевые компоненты (ридер, редактор, карточка документа, навигационная панель, модальные окна) являются переиспользуемыми. Такой подход обеспечивает единообразие интерфейса, упрощает поддержку и расширение функциональности.

2.9 Обоснование выбора инструментария

На основе сформулированных решений и спроектированной архитектуры выполним выбор ключевых технологий для реализации платформы.

2.9.1 Выбор технологий серверной части

Для реализации серверной части рассмотрим три популярных стека: Java + Spring Boot, Python + Django и Node.js + Express. Сравнительный анализ технологий представлен в таблице 9.

Таблица 9 – Сравнительный анализ фреймворков для бэкенд-разработки

Критерий	Java + Spring Boot	Python + Django	Node.js + Express
Производительность	Высокая (компилируемый язык, JIT-оптимизации)	Средняя (интерпретируемый)	Высокая (асинхронная модель)
Типизация	Статическая, строгая	Динамическая	Динамическая
Экосистема	Огромная (maven)	Большая	Большая (npm)
Безопасность	Встроенная (Spring Security)	Встроенная (Django Security)	Требует дополнительных библиотек
Поддержка многопоточности	Отличная (многопоточность на уровне языка)	Ограниченная (GIL)	Заменяется моделью асинхронности
Скорость вычислений	Высокая	Средняя	Средняя

В результате сравнения фреймворков был выбран Spring Boot + Java. Предпочтение отдано ввиду высокой производительности при вычислительных операциях (парсинг документа, вычисление хэшей, подпись HMAC-

SHA256), строгой статической типизации, снижающей вероятность ошибок в сложной доменной модели, и встроенным механизмам безопасности.

Для организации хранения данных сравним реляционный и нереляционный варианты на примере наиболее популярных СУБД: PostgreSQL и MongoDB (см. таблицу 10).

Таблица 10 – Сравнение реляционной и документоориентированной СУБД

Критерий	PostgreSQL (реляционная)	MongoDB (документоориентированная)
Поддержка связей между сущностями	Естественная (FK, JOIN)	Эмулируется в коде
ACID-транзакции	Полная поддержка	Ограниченная (на уровне документа)
Сложные запросы с агрегацией	Мощности SQL	Ограниченные возможности
Поддержка UUID	Встроенная	Требуется дополнительная настройка
Целостность данных	Высокая (ссылочная целостность)	Не гарантируется

PostgreSQL обеспечивает ссылочную целостность между таблицами user, document, publication и favorites, что критически важно для консистентности данных платформы. Поэтому в качестве основной базы данных выбрана реляционная СУБД PostgreSQL.

Для временного хранения refresh-токенов выбрана СУБД Redis как высокопроизводительное хранилище «ключ-значение» с встроенным механизмом TTL.

2.9.2 Выбор технологий клиентской части

Для вычислительно емких операций (парсинг, валидация, верификация документов .doco) на клиентской стороне планируется использовать модуль на Rust, компилируемый в WebAssembly. Это позволяет разгрузить сервер и выполнять обработку файлов с производительностью, близкой к нативной [8].

Для реализации клиентской части рассмотрим три наиболее популярные фронтенд-платформы: React, Angular и Vue.js. Их сравнительный анализ представлен в таблице 11.

Таблица 11 – Сравнительный анализ инструментов для фронтенд-разработки

Критерий	React	Angular	Vue.js
Архитектурный подход	Компонентный (CBA)	MVVM	MVVM
Скорость разработки	Высокая	Средняя	Очень высокая
Производительность	Высокая	Высокая	Высокая
Гибкость и уровень контроля	Высокий	Ограниченный	Средний
Экосистема и сообщество	Огромная	Большая	Средняя
Пригодность для кастомных компонентов (ридер, редактор)	Отличная	Хорошая	Хорошая

В результате анализа был выбран React благодаря высокой гибкости и компонентному подходу, позволяющему реализовать специализированные компоненты для рендеринга интерактивных блоков (вкладки, степпер, аккордеон, тест). Большая экосистема обеспечивает доступ к проверенным решениям для маршрутизации (React Router) и управления состоянием.

В качестве языка разработки был выбран TypeScript. Это обусловлено тем, что для строгого соответствия спецификации формата .dts целесообразно использовать язык со статической типизацией и контролем типов.

2.9.3 Выбор инфраструктурных решений

Для контейнеризации выбран Docker с оркестрацией через Docker Compose, что обеспечивает воспроизводимость окружения и упрощает развертывание.

В качестве веб-сервера и обратного прокси планируется использовать Nginx.

Для управления исходным кодом проекта используется распределенная система контроля версий Git. В качестве удаленного хостинга репозитория выбран GitHub, что обеспечивает централизованное хранение кода и удобный веб-интерфейс.

3 РЕАЛИЗАЦИЯ ПРОГРАММНОГО ОБЕСПЕЧЕНИЯ

3.1 Организация исходного кода и структура проекта

Исходный код платформы Doceum организован в виде монорепозитория, объединяющего бэкенд, фронтенд и библиотеки в одном репозитории (рисунок 15). Такой подход упрощает управление зависимостями, согласование изменений между клиентской и серверной частями, а также обеспечивает единую историю версий.

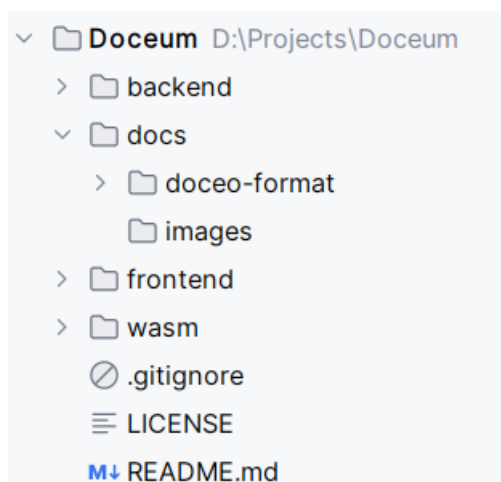


Рисунок 15 – Структура монорепозитория с исходным кодом проекта

Бэкенд организован по принципу модульного монолита. Каждый функциональный блок выделен в отдельный модуль с четкими границами ответственности (рисунок 16). В составе платформы реализованы следующие модули:

- auth – аутентификация и управление пользователями,
- documents – создание, редактирование, публикация и верификация документов,
- hub – каталог публикаций и поиск,
- profile – личный кабинет пользователя, избранное.

Модули слабо связаны между собой: взаимодействие осуществляется через интерфейсы, вынесенные в common/ports. Это позволяет при

необходимости вынести модуль в отдельный микросервис без изменения остальной системы.

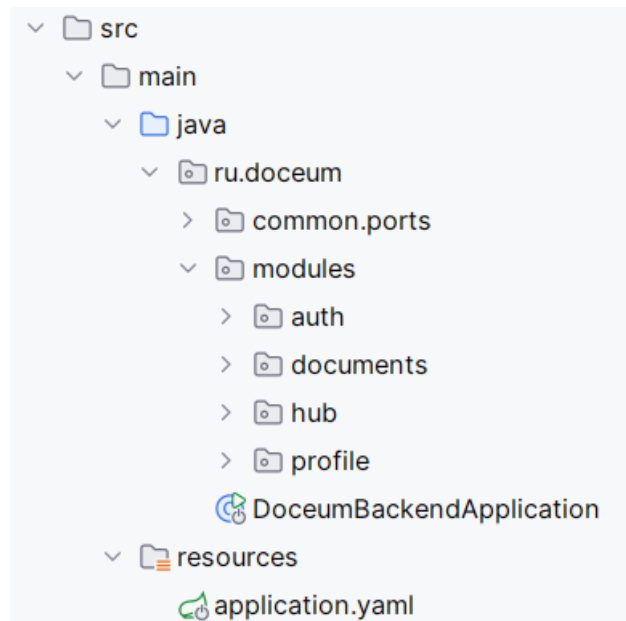


Рисунок 16 – Модульная структура серверной части

Внутри каждого модуля код организован согласно принципам чистой архитектуры (рисунок 17), обеспечивающей независимость бизнес-логики от фреймворков и библиотек.

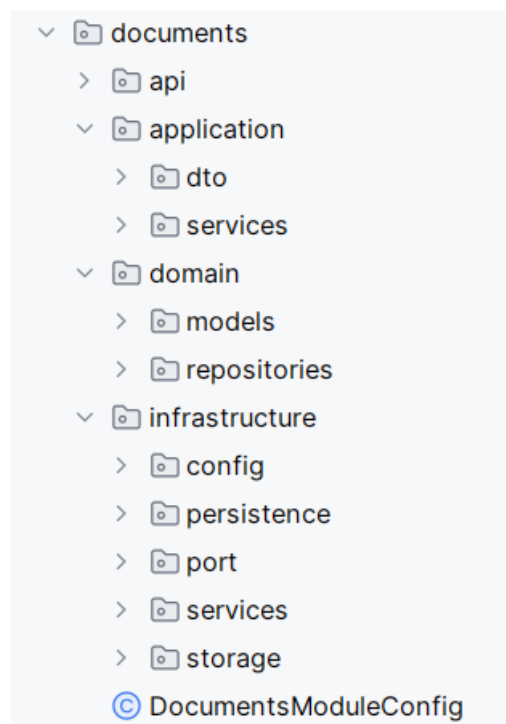


Рисунок 17 – Структура чистой архитектуры внутри модуля серверной части

Фронтенд организован как Single Page Application с компонентной (СВА) архитектурой. Структура папок (см. рисунок 18) отражает разделение на компоненты, страницы, хранилища состояния (MobX) и API-клиенты.

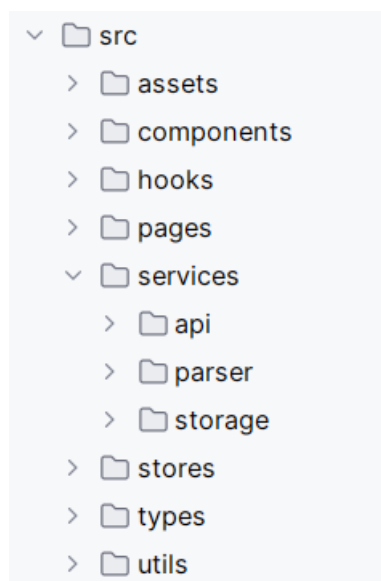


Рисунок 18 – СВА-структура клиентской части

Такая организация кода обеспечивает низкую связанность, высокую тестируемость и удобство дальнейшего сопровождения системы.

3.2 Реализация слоя данных

Хранение данных в платформе Doseum организовано с использованием комбинации реляционной базы данных PostgreSQL, высокопроизводительного хранилища Redis и файловой системы. Такой подход позволяет эффективно управлять структурированными метаданными, временными токенами и бинарными файлами документов.

Основным хранилищем структурированных данных выступает PostgreSQL, схема которого включает таблицы user (учетные записи пользователей), document (метаданные черновиков), publication (опубликованные версии документов с подписью) и favorites (избранные документы пользователей). Физическая схема базы данных представлена на рисунке 19.

Для работы с базой данных используется Spring Data JPA – реализация спецификации Java Persistence API – представляющая собой ORM (Object-

Relational Mapping) и позволяющая отображать Java-объекты на таблицы реляционной базы данных без написания низкоуровневых SQL-запросов.

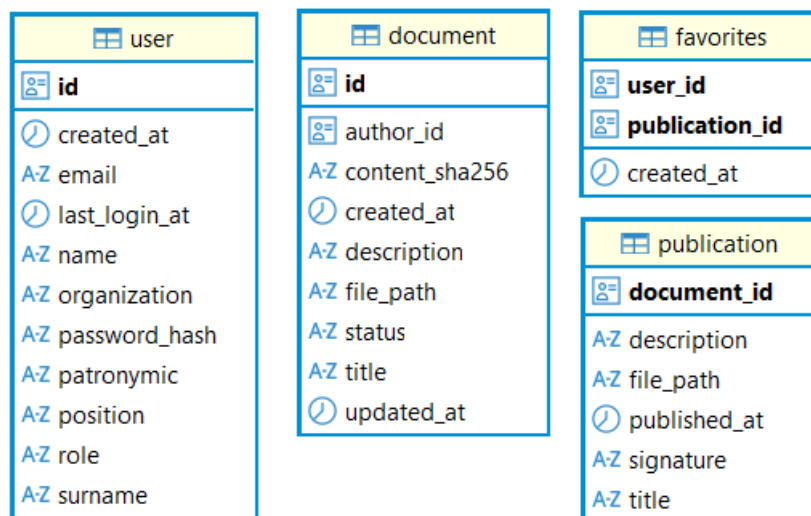


Рисунок 19 – Физическая схема базы данных, созданной при помощи ORM

JPA-сущности (UserEntity, DocumentEntity, PublicationEntity, FavoriteEntity) описывают структуру таблиц и связи между ними с помощью аннотаций (@Entity, @Id, @ManyToOne и др. – см. рисунок 20), а интерфейсы репозитория (UserRepository, DocumentRepository) агрегируют JpaRepository, получая стандартные CRUD-операции и возможность написания кастомных запросов.

```

11     @Entity & Honsage
12     @Table(name = "favorites")
13     @Getter
14     @Setter
15     @NoArgsConstructor
16     public class FavoriteEntity {
17
18         @EmbeddedId
19         private FavoriteId id;
20
21         @Column(name = "created_at", nullable = false)
22         private Instant createdAt;
23

```

Рисунок 20 – Пример описания таблицы классом при помощи JPA аннотаций

Refresh-токены не сохраняются в основной базе данных, а хранятся в Redis – высокопроизводительном хранилище «ключ-значение». Это обеспечивает быстрый доступ к токенам и их автоматическое удаление по истечении времени жизни (TTL = 30 дней). Ключи организованы по схеме `refresh:{token_hash}`, также поддерживается множество активных токенов пользователя по ключу `user_tokens:{user_id}` (см. рисунок 21).

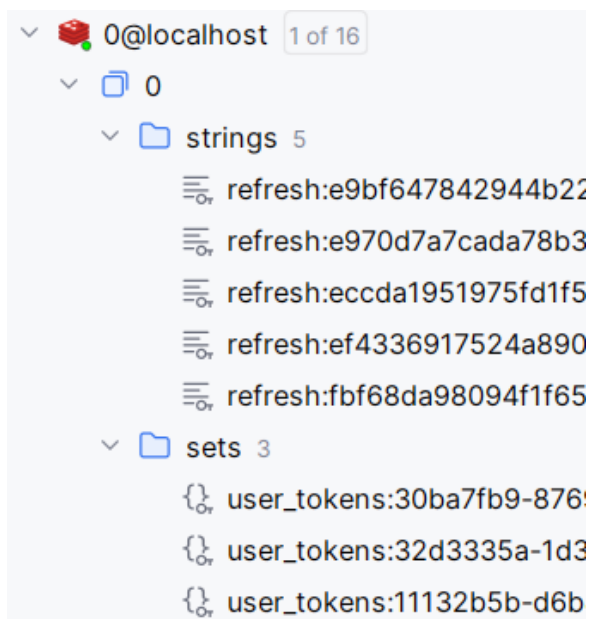


Рисунок 21 – Структура ключей Redis

Настройка подключения к PostgreSQL и Redis вынесена в файл `application.yaml`, причем параметры подключения (хост, порт, имя базы данных, логин, пароль) задаются через переменные окружения, что соответствует принципам 12-факторного приложения [9].

Бинарные файлы документов в формате `.docseo` хранятся в файловой системе сервера. Организация директорий имеет структуру `{storage_root}/drafts/{user_id}/{document_id}.docseo` для черновиков и `{storage_root}/publications/{document_id}.docseo` для публикаций (см. рисунок 22), что обеспечивает изоляцию черновиков по пользователям и прямой доступ к публикациям.

Класс `FileStorageService` предоставляет методы для сохранения, чтения и удаления файлов, а также вычисления контрольных сумм (SHA-256).



Рисунок 22 – Структура файлового хранилища

3.3 Реализация модуля аутентификации и авторизации

Модуль аутентификации и авторизации отвечает за регистрацию пользователей, вход в систему, управление сессиями и разграничение доступа к ресурсам платформы. В основе модуля лежит использование JWT (JSON Web Token) для access-токенов и Redis для хранения refresh-токенов, что обеспечивает короткоживущие сессии с возможностью продления без повторного ввода пароля.

При регистрации пароль хэшируется с помощью BCrypt и сохраняется в таблицу user. При успешном входе AuthApplicationService.login() генерирует пару токенов: access-токен (JWT, время жизни 15 минут) и refresh-токен (случайная строка, 30 дней). Access-токен создается JwtService.generateAccessToken() с добавлением в claims идентификатора пользователя, email и роли. Refresh-токен сохраняется в Redis.

JwtAuthenticationFilter перехватывает запросы, извлекает токен из заголовка Authorization, проверяет его через JwtService.validateToken() и при успехе устанавливает аутентификацию в SecurityContextHolder. Обновление токенов выполняется через эндпоинт /api/auth/refresh с проверкой refresh-токена в Redis, после чего старая пара заменяется новой (ротация). При выходе из системы refresh-токен удаляется из Redis.

Права доступа настроены в SecurityConfig: /api/auth/** открыт для всех, просмотр публикаций доступен без аутентификации, работа с каталогом требует наличия зарегистрированной роли (см. рисунок 23). Ролевая модель включает в себя **READER**, **AUTHOR** и **ADMIN**.

```
.sessionManagement(  
    SessionManagementConfigurer<HttpSecurity> session ->  
        session.sessionCreationPolicy(SessionCreationPolicy.STATELESS)  
)  
.authorizeHttpRequests( AuthorizationManagerRequestMat... auth -> auth  
    .requestMatchers(🔒 "/api/auth/**").permitAll()  
    .requestMatchers(🔒 "/api/public/**").permitAll()  
    .requestMatchers(🔒 "/api/hub/**").hasAnyRole( ...roles: "READER", "AUTHOR", "ADMIN")  
    .requestMatchers(🔒 "/api/documents/*/view").permitAll()  
    .requestMatchers(🔒 "/api/documents/verify").permitAll()  
    .anyRequest().authenticated()  
)  
.addFilterBefore(jwtAuthenticationFilter, UsernamePasswordAuthenticationFilter.class);
```

Рисунок 23 – Настройка прав доступа к эндпоинтам

3.4 Реализация модуля управления документами

Модуль управления документами обеспечивает создание, редактирование, публикацию и верификацию документов формата .doseo. Взаимодействие осуществляется через REST-контроллер DocumentController, который делегирует операции сервисному слою DocumentApplicationService.

В основе модуля лежат две сущности: Document (черновик) и Publication (публикация). Черновик может находиться в статусах NOT_PUBLISHED, PUBLISHED или ARCHIVED. Публикация является отдельной сущностью, поскольку подписанный файл не должен изменяться при последующем редактировании черновика.

При создании документа сервис генерирует минимальный валидный файл .doseo, сохраняет его в директорию черновиков и записывает метаданные в базу данных (см. рисунок 24). При публикации файл копируется в директорию публикаций, подписывается с использованием HMAC-SHA256, после чего создается запись публикации с подписью и датой. Статус документа меняется на PUBLISHED. При повторной публикации создается новая версия.



```

30 public class DocumentApplicationService {
40     @Transactional 2 usages Honsage *
41     @ public UUID createDocument(UUID authorId, CreateDocumentRequest request) {
42         UUID documentId = UUID.randomUUID();
43         byte[] minimalDoceo = doceoGenerator.generateMinimal(
44             request.getTitle(), documentId, authorId
45         );
46         String filePath = fileStorageService.saveDraft(authorId, documentId, minimalDoceo);
47         String contentSha256 = doceoParser.calculateContentSha256(minimalDoceo);
48
49         Document document = Document.createNew(
50             documentId, authorId, request.getTitle(), request.getDescription(),
51             filePath, contentSha256
52         );
53         documentRepository.save(document);
54         return documentId;
55     }

```

Рисунок 24 – Создание черновика документа в сервисе модуля documents

Просмотр опубликованного документа доступен по публичной ссылке без аутентификации. При загрузке файла читатель может инициировать верификацию: сервис пересчитывает подпись и сравнивает с сохраненной. Снятие с публикации удаляет запись публикации и переводит документ в статус черновика. Удаление документа переводит его в статус ARCHIVED.

3.5 Реализация парсера формата документа

Для обработки файлов .doceo на клиентской стороне разработана библиотека на языке Rust, компилируемая в WebAssembly. Выбор Rust обусловлен производительностью при распаковке ZIP-архивов и валидации JSON-схем, а также безопасностью памяти. WASM-пакет интегрируется в React-приложение и вызывается при загрузке документа.

Библиотека построена модульно (см. рисунок 25). Модуль archive распаковывает ZIP и извлекает manifest.json и content.json. Модули manifest и content описывают структуры данных документа. validator проверяет целостность и соответствие схемам. signature верифицирует HMAC-SHA256 подпись. Директория nodes содержит описание типов узлов.

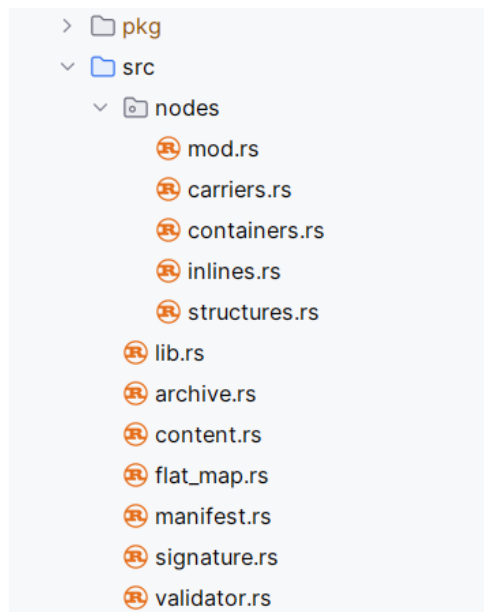


Рисунок 25 – Структура исходного кода библиотеки парсера

Основная функция `parse_document_inner` (рисунок 26) последовательно выполняет распаковку, десериализацию манифеста и содержимого. При ошибке возвращается структура с ее описанием. При успехе – дерево узлов документа.

```

lib.rs x
46 fn parse_document_inner(bytes: &[u8]) -> ParseResult {
47     let extracted = match archive::extract(bytes) {
48         Ok(e) => e,
49         Err(e) => return ParseResult::Err(ParseErr {
50             errors: vec![validator::ValidationError::archive(e)],
51         }),
52     };
53
54     let manifest: manifest::Manifest =
55         match serde_json::from_slice(&extracted.manifest_bytes) {
56             Ok(m) => m,
57             Err(e) => return ParseResult::Err(ParseErr {
58                 errors: vec![validator::ValidationError::deserialise("manifest.json", e)],
59             }),
60         };
61
62     let document: content::ContentDocument =
63         match serde_json::from_slice(&extracted.content_bytes) {
64             Ok(d) => d,
65             Err(e) => return ParseResult::Err(ParseErr {
66                 errors: vec![validator::ValidationError::deserialise("content.json", e)],
67             }),
68         };

```

Рисунок 26 – Код для парсинга содержимого архива .docseo

3.6 Реализация клиентской части

Клиентская часть платформы Doseum реализована как одностраничное приложение (SPA) на React с использованием TypeScript. Управление состоянием выполняется с помощью MobX, маршрутизация – через React Router. Интерфейс построен на компонентном подходе с изолированными стилями (CSS Modules).

Приложение содержит страницы: вход, регистрация, каталог публикаций, просмотр документа (по ID и локальный файл), редактор документа, личный кабинет. Защищенные маршруты обернуты в `ProtectedRoute`, который проверяет наличие токена и роль пользователя. Настройка роутинга представлена на рисунке 27.

```
<Routes>
  <Route element={<MainLayout />}>
    <Route path="/" element={<HomePage />} />
    <Route path="/viewer/local" element={<LocalViewerPage />} />
    <Route path="/viewer/local/preview" element={<LocalPreviewPage />} />

    // Для любых аутентифицированных пользователей
    <Route element={<ProtectedRoute />}>
      <Route path="/library" element={<LibraryPage />} />
      <Route path="/profile" element={<ProfilePage />} />
      <Route path="/viewer/:id" element={<ViewerPage />} />
    </Route>

    // Для авторов
    <Route element={<ProtectedRoute allowedRoles={['AUTHOR', 'ADMIN']} />}>
      <Route path="/editor" element={<EditorPage />} />
      <Route path="/editor/:documentId" element={<EditorPage />} />
    </Route>
  </Route>

  <Route element={<AuthLayout />}>
    <Route path="/login" element={<LoginPage />} />
    <Route path="/register" element={<RegisterPage />} />
  </Route>
</Routes>
```

Рисунок 27 – Настройка роутинга в SPA

Ключевой компонент платформы – `EditorPage`. Он разделен на три функциональные зоны: палитра блоков (содержит доступные типы блоков для добавления), холст (canvas) с редактируемыми блоками документа и панель свойств выбранного блока. Такая компоновка повторяет логику работы

профессиональных конструкторов контента (Tilda, Notion) и привычна для авторов. Состояние редактора централизованно управляется EditorStore (MobX), который хранит карту блоков (Map<id, Block>) и массив корневых идентификаторов, обеспечивающий порядок отображения. Для редактирования форматированного текста используется RichTextEditor (см. рисунок 28) на базе TipTap редактора текста с поддержкой расширений (жирный, курсив, подчеркнутый, зачеркнутый, ссылки, моноширинный код). TipTap выбран за гибкость и возможность кастомизации поведения под специфику формата .docso.

```
frontend > src > pages > editor > components > RichTextEditor.tsx > RichTextEditor > editor > onUpdate
64 export const RichTextEditor = ({ content, onChange, placeholder }: RichTextEditorProps) => {
65   const [activeStates, setActiveStates] = useState({
66     bold: false,
67     italic: false,
68     underline: false,
69     strike: false,
70     code: false,
71     link: false,
72   });
73
74   const editor = useEditor({
75     extensions: [
76       StarterKit.configure({
77         heading: false, // Отключаем заголовки
78         bulletList: false, // Отключаем списки
79         orderedList: false, // Отключаем списки
80         codeBlock: false, // Отключаем блоки кода
81       }),
82       Underline,
83       Strike,
84       Link.configure({
85         openOnClick: false,
86         HTMLAttributes: { class: styles.link },
87       }),
88       Placeholder.configure({
89         placeholder: placeholder || 'Напишите что-нибудь...',
90         emptyEditorClass: styles.emptyEditor,
91       }),
92       TabHandler,
93     ],
```

Рисунок 28 – Фрагмент реализации компонента RichTextEditor

Управление авторизацией централизовано в AuthStore (см. рисунок 29). Хранилище содержит наблюдаемые поля user, isAuthenticated, accessToken и refreshToken, а также методы для входа, регистрации, выхода и обновления токенов. При загрузке приложения AuthStore проверяет наличие токенов в localStorage и, если access-токен истек, автоматически выполняет его

обновление через `/api/auth/refresh`. Все исходящие запросы к API проходят через перехватчик, который добавляет заголовок `Authorization: Bearer <token>`.

```
frontend > src > stores > TS AuthStore.ts > AuthStore
6  class AuthStore {
67    async login(credentials: LoginRequest): Promise<boolean> {
68      this.setLoading(true);
69      this.setError(null);
70
71      try {
72        const response = await authApi.login(credentials);
73
74        runInAction(() => {
75          storageService.setAccessToken(response.accessToken);
76          storageService.setRefreshToken(response.refreshToken);
77          this.setUser(response.user);
78          this.isLoading = false;
79        });
80
81        return true;
82      } catch (err) {
83        runInAction(() => {
84          this.error = err instanceof Error ? err.message : 'Login failed';
85          this.isLoading = false;
86        });
87        return false;
88      }
89    }
}
```

Рисунок 29 – Фрагмент реализации AuthStore

3.7 Конфигурация сетевого взаимодействия

При разработке фронтенд и бэкенд Досеум работают на разных портах (`localhost:3000` и `localhost:8080`), что приводит к кросс-доменным запросам, блокируемым браузером. Для решения этой проблемы кастомно настраивалась CORS.

На бэкенде в `SecurityConfig` определена конфигурация CORS: разрешенные источники вынесены в переменную окружения `CORS_ALLOWED_ORIGINS`, что позволяет для разработки указать `http://localhost:3000`, а для продакшена – реальный домен фронтенда. Также разрешены все необходимые HTTP-методы и заголовки (см. рисунок 30).

```

@Value("${cors.allowed-origins}")
private String allowedOrigins;

@Bean
public CorsConfigurationSource corsConfigurationSource() {
    CorsConfiguration configuration = new CorsConfiguration();

    configuration.setAllowedOrigins(List.of(allowedOrigins.split(", ")));

    configuration.setAllowedMethods(List.of("GET", "POST", "PUT", "DELETE", "OPTIONS"));
    configuration.setAllowedHeaders(List.of("Authorization", "Content-Type", "X-Refresh-Token"));
    configuration.setAllowCredentials(true);
    configuration.setExposedHeaders(List.of("Authorization", "X-Refresh-Token"));
    configuration.setMaxAge(3600L);

    UrlBasedCorsConfigurationSource source = new UrlBasedCorsConfigurationSource();
    source.registerCorsConfiguration(pattern: "**", configuration);
    return source;
}

```

Рисунок 30 – Настройка конфигурации CORS на бэкенде

Для взаимодействия с бэкендом создан единый API-клиент (см. рисунок 31). Его базовый URL берется из переменной окружения VITE_API_URL. Перехватчик запросов автоматически добавляет JWT-токен в заголовок Authorization, а перехватчик ответов при получении 401 инициирует обновление токена через /api/auth/refresh.

```

frontend > src > services > api > TS client.ts > request > makeRequest
19 | const API_BASE = import.meta.env.VITE_API_URL;
20 |
21 | async function request<T>(endpoint: string, options: RequestOptions = {}): Promise<T> {
22 |     if (!endpoint || endpoint.trim() === '') {
23 |         throw new ApiError(400, 'Invalid endpoint');
24 |     }
25 |
26 |     const { requireAuth = false, ...fetchOptions } = options;
27 |
28 |
29 |     const getHeaders = (token?: string): HeadersInit => {
30 |         const headers: HeadersInit = {
31 |             'Content-Type': 'application/json',
32 |         };
33 |
34 |         if (fetchOptions.headers) {
35 |             Object.assign(headers, fetchOptions.headers);
36 |         }
37 |
38 |         if (requireAuth && token) {
39 |             headers['Authorization'] = `Bearer ${token}`;
40 |         }

```

Рисунок 31 – API-клиент на фронтенде

3.8 Результаты разработки

В результате реализации разработана платформа Doseum, включающая серверную часть на Spring Boot/Java, клиентскую часть на React/TypeScript, модуль парсера на Rust/WASM, а также базы данных PostgreSQL и Redis. Реализованы следующие ключевые функции:

- регистрация и аутентификация пользователей с ролевой моделью (READER, AUTHOR, ADMIN);
- создание, редактирование и публикация интерактивных документов в формате .doseo;
- просмотр опубликованных документов и загрузка локальных файлов;
- верификация подписи документов (HMAC-SHA256);
- каталог публикаций с поиском и избранным.

Примеры внешнего вида интерфейса платформы представлены на рисунках 32 и 33 (страница редактирования и просмотра соответственно).

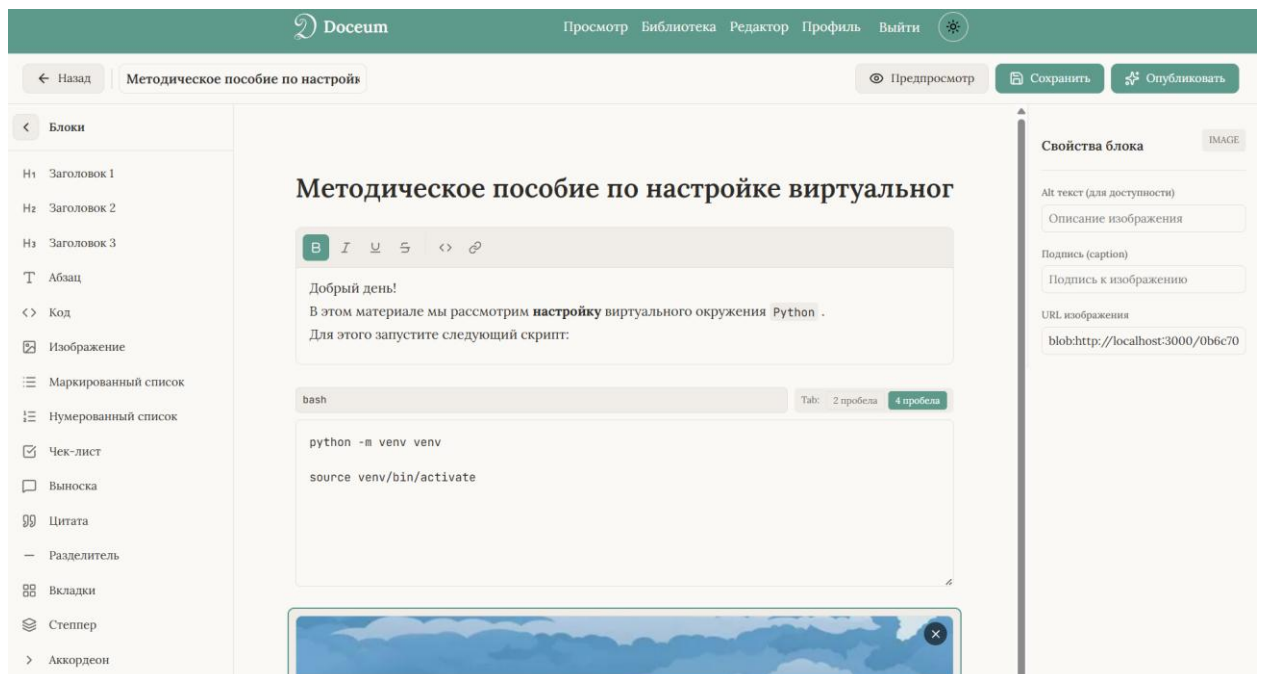


Рисунок 32 – Скриншот интерфейса страницы редактирования документов

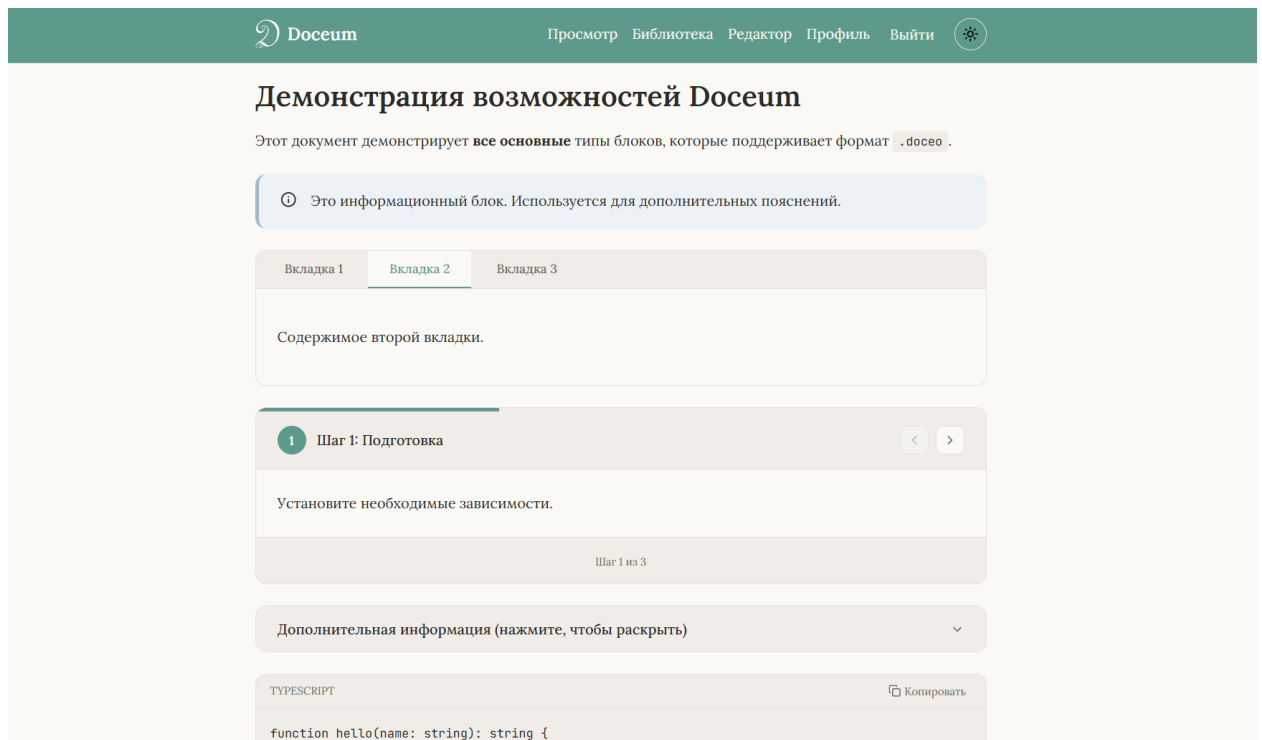


Рисунок 33 – Скриншот интерфейса страницы просмотра документов

4 ТЕСТИРОВАНИЕ И РАЗВЕРТЫВАНИЕ

4.1 Тестирование программного обеспечения

4.1.1 Модульное тестирование

Для проверки корректности работы ключевых компонентов серверной и клиентской частей платформы организовано модульное тестирование. Основной его целью является выявление ошибок на ранних этапах разработки и обеспечение стабильности системы при внесении изменений.

Модульное тестирование бэкенда реализовано с использованием фреймворка JUnit 5 в связке с Mockito (для изоляции зависимостей) и Spring Boot Test для интеграционного тестирования контроллеров. Конфигурация тестового окружения выполнена с использованием H2 – встраиваемой базы данных [10], которая запускается в памяти при каждом прогоне тестов. Это обеспечивает изоляцию тестов друг от друга и отсутствие влияния на основную базу данных.

Пример тестирования проверки регистрации представлен на рис. 34.



```
31 class AuthServiceTest {
78     @Test
79     void testRegisterSuccess() {
80         when(userRepository.existsByEmail(any())).thenReturn(false);
81         when(passwordEncoder.encode(rawPassword: "password123")).thenReturn("encodedPassword");
82         when(jwtService.generateAccessToken(any(User.class))).thenReturn("accessToken");
83         when(refreshTokenService.createRefreshToken(any())).thenReturn("refreshToken");
84         when(userRepository.save(any(User.class))).thenReturn(mockUser);
85
86         AuthResponse response = authService.execute(registerRequest);
87
88         assertNotNull(response);
89         assertNotNull(response.getAccessToken());
90         assertNotNull(response.getRefreshToken());
91         assertNotNull(response.getUser());
92         assertEquals("test@example.com", response.getUser().getEmail());
93         assertEquals("READER", response.getUser().getRole());
94         assertEquals("Test", response.getUser().getSurname());
95         assertEquals("User", response.getUser().getName());
96     }
}
```

Рисунок 34 – Модульный тест проверки регистрации для AuthService

Полный перечень тестируемых компонентов бэкенда представлен в таблице 12.

Таблица 12 – Перечень групп модульных тестов для бэкенда

Модуль	Тестируемые компоненты	Количество тестов
Auth	AuthService: регистрация, логин, refresh token, logout	12
Auth	User, Email, Password – доменные модели и value objects	13
Documents	DocumentApplicationService: создание, сохранение, удаление документа	8
Documents	Document – доменная модель (статусы, обновление)	7
Profile	ProfileApplicationService: избранное (добавление, удаление)	12
Hub	HubApplicationService: поиск по названию/автору, пагинация	19
Hub	HubController: REST эндпоинты, параметры, авторизация	11

Все модульные тесты для бэкенда прошли успешно (см. рисунок 35).

```
[INFO] Results:
[INFO]
[INFO] Tests run: 82, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 30.161 s
[INFO] Finished at: 2026-05-18T01:36:13+03:00
[INFO] -----
```

Рисунок 35 – Отчет об успешном прохождении модульных тестов на бэкенде

Модульное тестирование фронтенда выполнено с использованием Vitest и Testing Library. MobX store тестируются в изоляции, а React-компоненты – с рендерингом в виртуальном DOM.

На рисунке 36 представлен фрагмент модульного теста для MobX-хранилища AuthStore, проверяющий корректность сохранения данных пользователя при успешной аутентификации. Тест имитирует вызов API-метода login с валидными учетными данными и проверяет, что после получения ответа от сервера в localStorage сохраняются accessToken, refreshToken и объект пользователя с правильными полями. Использование моков (jest.mock) позволяет

изолировать тестируемую логику от реальных HTTP-запросов, что делает тест детерминированным и быстрым.

```
15 describe( name: 'AuthStore', () :void => {  Honsage
29   it( name: 'setUser сохраняет пользователя', () :void => {
30     const user = {
31       id: '123',
32       email: 'test@example.com',
33       role: 'READER' as const,
34       surname: 'Test',
35       name: 'User',
36     };
37
38     authStore.setUser(user);
39     expect(authStore.user).toEqual(user);
40     expect(storageService.getUser()).toEqual(user);
41   });
```

Рисунок 36 – Модульный тест проверки сохранения данных в AuthStore

На рисунке 37 приведен отчет об успешном выполнении модульных тестов клиентской части. Тесты охватывают MobX-хранилища (AuthStore, DocumentStore, UiStore), утилиты валидации и React-компоненты рендеринга блоков (BlockRenderer, InlineRenderer, Tabs, Accordion). Каждый тест включает проверку позитивных и негативных сценариев, а также граничных случаев.

```
✓ tests/unit/components/BlockRenderer.test.tsx (5 tests) 44ms
✓ tests/unit/components/blocks/Accordion.test.tsx (2 tests) 170ms
✓ tests/unit/components/blocks/Tabs.test.tsx (3 tests) 179ms

Test Files  8 passed (8)
Tests      48 passed (48)
Start at   00:41:32
Duration   3.02s (transform 1.06s, setup 2.98s, import 2.77s, tests 561ms, environment 11.23s)
```

Рисунок 37 – Отчет об успешном прохождении модульных тестов фронтенда

На рисунке 38 представлен анализ покрытия кода фронтенда тестами, сгенерированный инструментом v8 coverage через Vitest. Общее покрытие сервисов и хранилищ превышает 70%, что свидетельствует о достаточном уровне тестирования критической бизнес-логики.

services/storage		84.61%	22/26	90%	9/10	90.9%
stores		76.71%	112/146	60.52%	23/38	78.37%
utils		100%	15/15	100%	0/0	100%

Рисунок 38 – Отчет о покрытии ключевого кода фронтенда тестами

4.1.2 Фаззинг-тестирование

В целях выявления скрытых дефектов и уязвимостей было проведено фаззинг-тестирование (fuzz testing) путем подачи на вход компонентов случайных, граничных и неожиданных данных. Основное внимание уделено компонентам, обрабатывающим пользовательский ввод: API-клиенту, сервису локального хранилища и утилитам форматирования.

Для фаззинг-тестирования использован фреймворк fast-check – библиотека для property-based тестирования. Fast-check автоматически генерирует тысячи случайных входных данных, мутирует их на основе структуры данных и ищет граничные случаи [11].

На рисунке 39 приведен пример фаззинг-теста для API клиента.

```

13  it( name: 'request не падает при любых эндпоинтах (50 итераций)', async () :Promise<void> => {
14      await fc.assert(
15          fc.asyncProperty(fc.string({ minLength: 0, maxLength: 100 }), async (endpoint :string )
16              mockFetch.mockResolvedValueOnce({
17                  ok: true,
18                  status: 200,
19                  json: async () :Promise<{}> => ({}),
20              });
21
22          try {
23              await request(endpoint, { requireAuth: false });
24              expect( actual: true ).toBe( expected: true );
25          } catch (err) {
26              if (endpoint === '' || endpoint.trim() === '') {
27                  expect(err).toBeInstanceOf(ApiError);
28                  expect(err.status).toBe( expected: 400 );
29              } else {
30                  throw err;
31              }
32          }
33      },
34      { numRuns: 50 }
35  );
36  });

```

Рисунок 39 – Фаззинг-тестирование API клиента

В ходе фаззинг-тестирования было обнаружено 4 дефекта, которые не были выявлены при обычном модульном тестировании (см. рисунок 40).

```
FAIL tests/fuzz/api-client.fuzz.test.ts > Fuzz: API Client > request не падает при любых заголовках (50 итераций)
Error: Property failed after 1 tests
{ seed: -947150406, path: "0:0", endOnFailure: true }
Counterexample: [{}]
Shrunk 1 time(s)

[4/4]

Test Files  3 failed | 6 passed (9)
Tests      4 failed | 14 passed (18)
Start at   23:28:32
Duration   6.11s (transform 809ms, setup 5.45s, import 18.17s, tests 1.39s, environment 17.98s)
```

Рисунок 40 – Отчет о неуспешном прохождении фаззинг-тестирования

Дефекты заключались в отсутствии валидации некорректных эндпоинтов в API клиенте, некорректное копирование пустых заголовков, возврат пустой строки вместо null при получении токена, а также ошибка форматирования даты при годе из шести цифр. После внесения исправлений все фаззинг тесты прошли успешно (см. рисунок 41).

```
Test Files  9 passed (9)
Tests      18 passed (18)
Start at   23:42:28
Duration   5.04s (transform 858ms, setup 2.81s, import 17.51s, tests 1.33s, environment 13.62s)
```

Рисунок 41 – Отчет об успешном прохождении фаззинг-тестирования

Таким образом, фаззинг подтвердил свою эффективность: найдены и устранены проблемы обработки граничных случаев, что повысило устойчивость системы к некорректным входным данным.

4.2 Верификация продукта

Проведем верификацию разработанной платформы. Проверим соответствие реализованной функциональности требованиям, сформулированным в главе 1 (таблица 4). Ниже приведена таблица соответствия, в которой для каждого функционального требования указано, как именно оно реализовано в системе (см. таблицу 13).

Таблица 13 – Верификация функциональных требований

ID	Функция	Способ реализации
FR-1.1	Регистрация пользователя	Эндпоинт POST /api/auth/register. Пароль хэшируется BCrypt, пользователь сохраняется в БД
FR-1.2	Аутентификация пользователя	Эндпоинт POST /api/auth/login. Возвращаются JWT access-токен и refresh-токен (хранится в Redis 30 дней)
FR-1.3	Авторизация пользователя	Ролевая модель (READER, AUTHOR, ADMIN) через Spring Security с ограничением доступа к эндпоинтам
FR-2.1	Создание интерактивного документа	Эндпоинт POST /api/documents. Генерируется минимальный .docx, статус NOT_PUBLISHED
FR-2.2	Редактирование документа	Эндпоинт PUT /api/documents/{id}/draft. Файл сохраняется в файловое хранилище
FR-2.3	Публикация документа	Эндпоинт POST /api/documents/{id}/publish. Документ подписывается (HMAC-SHA256), статус PUBLISHED
FR-2.4	Снятие документа с публикации	Эндпоинт DELETE /api/documents/{id}/publish. Удаляется запись публикации, статус NOT_PUBLISHED
FR-3.1	Отображение опубликованного документа	Эндпоинт GET /api/documents/{id}/view. Файл парсится WASM, рендерится BlockRenderer
FR-3.2	Скачивание документа	Эндпоинт GET /api/documents/{id}/view с заголовком Content-Disposition: attachment
FR-3.3	Поиск опубликованных документов	Эндпоинты /api/hub/recent и /api/hub/documents. Поиск по названию и автору с пагинацией
FR-3.4	Добавление документа в избранное	Эндпоинты POST/DELETE /api/profile/favorites/{publicationId}. Хранение в таблице favorites
FR-3.5	Отображение локального документа	Страница /viewer/local. Загрузка файла с диска, парсинг и рендер без отправки на сервер
FR-4.1	Подпись документа	DocoSigner вычисляет HMAC-SHA256 манифеста. Подпись сохраняется в publication.signature
FR-4.2	Проверка подписи документа	Эндпоинт POST /api/documents/verify. Возвращает verified, unsigned или tampered
FR-5.1	Управление публикациями	Автор удаляет документ (статус ARCHIVED) или снимает с публикации. Администратор имеет полный доступ
FR-5.2	Управление пользователями	Администратор изменяет роли и удаляет учетные записи

Все 16 функциональных требований, сформулированных в главе 1, успешно реализованы и подтверждены в ходе тестирования. Таким образом, платформа DocuSign обеспечивает полноценный жизненный цикл интерактивного документа: от первоначального создания и редактирования через визуальный редактор до публикации в каталоге, поиска, скачивания, офлайн-просмотра и криптографической верификации целостности и авторства. Реализованная ролевая модель (READER, AUTHOR, ADMIN) позволяет гибко

управлять правами доступа, обеспечивая как открытость каталога для читателей, так и безопасность процесса публикации для авторов.

4.3 Контейнеризация системы

Для упрощения развертывания и обеспечения воспроизводимости окружения платформа Досеум полностью контейнеризирована с использованием Docker. Все компоненты системы упакованы в контейнеры и управляются через Docker Compose.

Бэкенд-приложение упаковано в Dockerfile с многоступенчатой сборкой (см. рисунок 42): на первом этапе с использованием образа Maven с JDK 21 собирается JAR-файл, на втором этапе на основе образа Eclipse Temurin JRE формируется финальный образ с минимальным размером. Фронтенд-приложение собирается в образе Node.js, после чего статические файлы копируются в образ Nginx (рисунок 43), который выступает в качестве веб-сервера для их раздачи.

```
1 >> FROM maven:3.9-eclipse-temurin-21 AS builder
2
3 WORKDIR /build
4
5 COPY pom.xml .
6 RUN mvn dependency:go-offline -B
7
8 COPY src ./src
9 RUN mvn clean package -DskipTests
10
11 FROM eclipse-temurin:21-jre-alpine
12
13 WORKDIR /app
14
15 COPY --from=builder /build/target/backend-*.jar app.jar
16
17 RUN mkdir -p /storage/drafts /storage/publications
18
19 ENTRYPOINT ["java", "-jar", "app.jar"]
20
```

Рисунок 42 – Конфигурация Dockerfile для бэкенда

```

1  >> FROM node:20-alpine AS builder
2
3  WORKDIR /build
4
5  COPY package*.json ./
6  RUN npm ci
7
8  COPY . .
9
10 RUN npx vite build
11
12 FROM nginx:alpine
13
14 COPY --from=builder /build/dist /usr/share/nginx/html
15 COPY nginx/nginx.conf /etc/nginx/conf.d/default.conf
16
17 EXPOSE 80
18
19 CMD ["nginx", "-g", "daemon off;"]

```

Рисунок 43 – Конфигурация Dockerfile для фронтенда

Оркестрация контейнеров выполняется с помощью Docker Compose. Файл `docker-compose.yml` описывает все сервисы платформы: backend (Spring Boot приложение), frontend (Nginx со статикой), postgres (база данных) и redis (хранилище токенов).

Конфиденциальные параметры подключения (пароль БД, секретные ключи JWT и подписи, разрешенные источники CORS) передаются в контейнер бэкенда через переменные окружения (см. рисунок 44), что позволяет гибко настраивать систему для разных сред без пересборки образов.

На рисунке 45 изображен интерфейс Docker Desktop запуска контейнеров при помощи Docker Compose.

```

3      services:
27     backend:
28         build: ./backend
29         container_name: doceum-backend
30         environment:
31             DB_HOST: ${DB_HOST}
32             DB_PORT: ${DB_PORT}
33             DB_NAME: ${DB_NAME}
34             DB_USER: ${DB_USER}
35             DB_PASSWORD: ${DB_PASSWORD}
36             REDIS_HOST: ${REDIS_HOST}
37             REDIS_PORT: ${REDIS_PORT}
38             JWT_SECRET: ${JWT_SECRET}
39             CORS_ALLOWED_ORIGINS: ${CORS_ALLOWED_ORIGINS}
40             STORAGE_ROOT: ${STORAGE_ROOT}
41             DOCEO_SIGNING_SECRET: ${DOCEO_SIGNING_SECRET}
42         ports:
43             - "8080:8080"
44         volumes:
45             - storage_data:${STORAGE_ROOT}

```

Рисунок 44 – Передача переменных окружения в контейнер бэкенда

☐	▼	doceum	-	-	-
☐		postgres	5b76c9f87b65	postgres:16-alpine	5432:5432 ↗
☐		backend	29ea0b814cf1	doceum-backend	8080:8080 ↗
☐		frontend	b814064fc45d	doceum-frontend	80:80 ↗
☐		redis	5e3b82549d67	redis:7-alpine	6379:6379 ↗

Рисунок 45 – Результат запуска системы с помощью Docker Compose

4.4 Документирование продукта

4.4.1 Спецификация формата интерактивного документа

Формат .doceo является центральной частью платформы Doceum. Для обеспечения возможности реализации совместимых ридеров и редакторов сторонними разработчиками, а также для фиксации правил валидации и версионирования, разработана официальная спецификация формата. Спецификация включает:

- физическую структуру ZIP-архива (обязательное наличие manifest.json, content.json и опциональной директории media/);
- схему манифеста (метаданные документа, реестр ресурсов, контрольная сумма, подпись);
- схему содержимого (дерево узлов документа);
- классификацию типов узлов (семантические контейнеры, семантические структуры, носители контента, встроенные узлы);
- правила валидации (16 инвариантов, включая уникальность идентификаторов, допустимость вложений, ссылочную целостность);
- политику версионирования и совместимости (обратная совместимость при добавлении новых типов узлов).

Полная спецификация формата .doceo версии 1.0 доступна по адресу: <https://honsage.github.io/Doceum/>.

4.4.2 Руководство по развертыванию

Для запуска платформы Doceum достаточно наличия инструментов Docker и Docker Compose. Развертывание выполняется из корня репозитория командой `docker compose up -d`.

Перед запуском необходимо настроить переменные окружения в файле `.env`: `DB_NAME`, `DB_USER`, `DB_PASSWORD`, `DB_HOST`, `DB_PORT`, `REDIS_HOST`, `REDIS_PORT`, `JWT_SECRET`, `CORS_ALLOWED_ORIGINS`, `DOCEO_SIGNING_SECRET`, `STORAGE_ROOT`.

После запуска фронтенд доступен по адресу `http://localhost`, бэкенд – `http://localhost:8080/api`.

Полное руководство с описанием системных требований, пошаговой инструкции и типовых проблем размещено в файле `README.md` в корне репозитория по адресу <https://github.com/Honsage/Doceum>.

4.5 Настройка CI

Для автоматизации процессов сборки и тестирования платформы Doseum настроен конвейер непрерывной интеграции (Continuous Integration) на базе GitHub Actions. Конвейер запускается при каждом пуше в ветки master, а также при создании pull request в ветку master.

Конвейер состоит из трех последовательных задач (jobs). Задача test-backend запускает модульные тесты серверной части: устанавливается JDK 21, после чего выполняются тесты с помощью Maven. Задача test-frontend аналогично запускает тесты клиентской части: устанавливается Node.js 20, устанавливаются зависимости через npm ci и выполняются модульные тесты. Задача build зависит от успешного выполнения обеих тестовых задач и выполняет сборку приложения: бэкенд собирается в JAR-файл с помощью Maven, фронтенд собирается в статические файлы для последующего развертывания.

Все задачи успешно проходят при каждом изменении в репозитории, что подтверждает работоспособность системы на всех этапах разработки. Результат выполнения последних запусков конвейера представлен на рисунке 46.

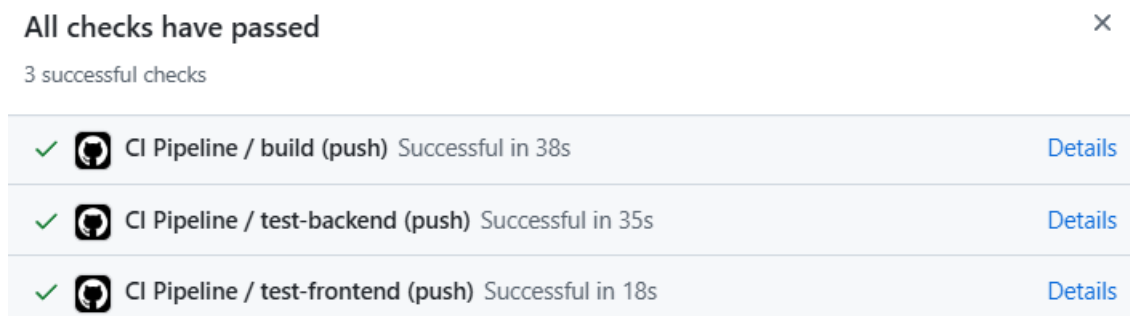


Рисунок 46 – Отчет об успешном прохождении задач CI

После завершения всех этапов разработки и тестирования кодовая база платформы Doseum была зафиксирована в репозитории и подготовлена к развертыванию в облачной среде. Внешний вид репозитория на GitHub представлен на рисунке 47.

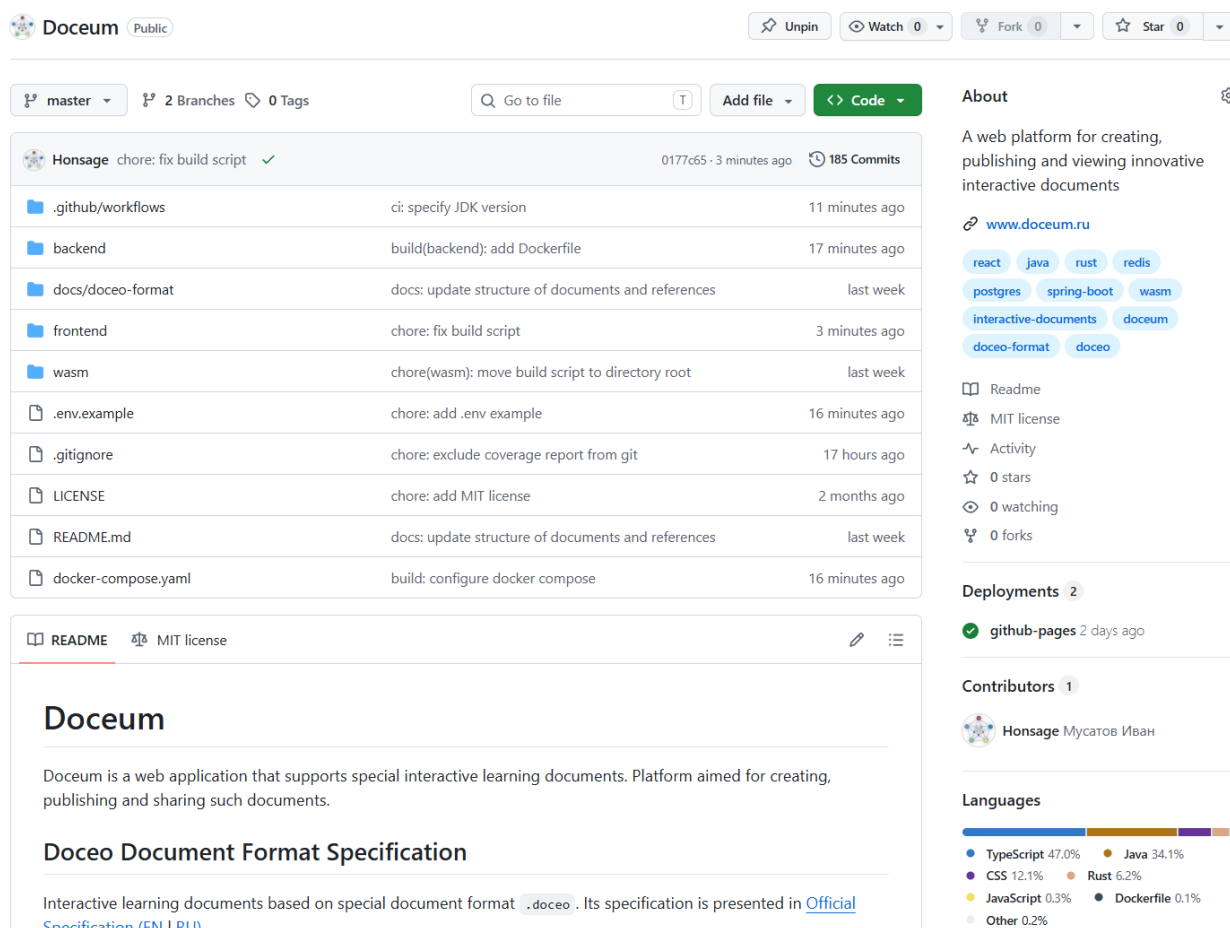


Рисунок 47 – Внешний вид репозитория Doceum на GitHub

4.6 Развертывание в облачной среде

Для обеспечения публичной доступности платформа Досеум развернута в облачной среде Cloud.ru. Выбор данного провайдера обусловлен наличием необходимых ресурсов (виртуальные машины, управление DNS-зонами), а также приемлемой стоимостью услуг для целей курсовой работы.

Развертывание выполнено на виртуальной машине под управлением операционной системы Linux. На виртуальную машину установлены Docker и Docker Compose, после чего из репозитория клонирован исходный код платформы, настроены переменные окружения и запущены контейнеры командой `docker compose up -d`. Скриншот созданной виртуальной машины в панели управления Cloud.ru представлен на рисунке 48.

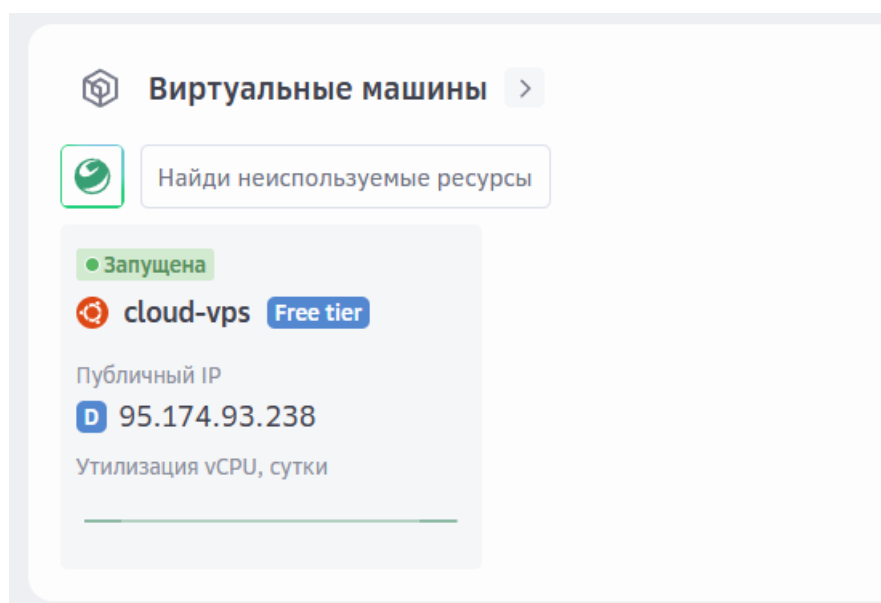


Рисунок 48 – Арендованная у провайдера Cloud.ru виртуальная машина

Для доступа к платформе по доменным именам `doceum.ru` и `www.doceum.ru` в DNS-зоне настроены А-записи, указывающие на публичный IP-адрес виртуальной машины. Скриншот с записями зон домена представлен на рисунке 49. После распространения DNS-изменений платформа доступна по указанным адресам (<https://doceum.ru>).

doceum.ru Активен Докуме				
Тип домена: Публичный				
Записи прямых зон				
↺ Поиск + Добавить				
Статус	Поддомен	Тип	Значения	TTL
Активна	@	SOA	evo-cns01.cloud.ru., admin.cloud...	3600
Активна	@	NS	evo-cns01.cloud.ru., evo-cns02.c...	3600
Активна	@	A	95.174.93.238	3600
Активна	www	A	95.174.93.238	3600

Рисунок 49 – А-записи, связывающие домен и ip адрес хоста

ЗАКЛЮЧЕНИЕ

В ходе выполнения курсовой работы была спроектирована и разработана платформа Doseum – клиент-серверное приложение для создания, публикации и распространения интерактивных образовательных материалов. Полученный продукт обеспечивает полный жизненный цикл интерактивного документа: от создания до верификации авторства и целостности.

На первом этапе работы был проведен анализ предметной области. Исследованы существующие решения, выявлены их ключевые недостатки: привязанность к платформе, отсутствие переносимости, разрыв между интерактивностью и автономностью документа. На основе анализа сформулированы требования к разрабатываемой системе.

На этапе проектирования разработана концепция продукта Doseum, включая ее фирменный стиль и целевую аудиторию. Обоснован выбор клиент-серверной архитектуры и веб-клиента как наиболее подходящих для поставленных задач. Построены функциональная модель в нотации IDEF0, архитектурные диаграммы в нотациях C4, UML, спроектирована база данных, спецификация программного и пользовательского интерфейсов, а также формата интерактивных документов .doseo.

На этапе реализации разработаны серверная часть, клиентское приложение и парсер формата документов. Реализованы регистрация и вход пользователей с разграничением прав, создание и редактирование черновиков, публикация документов с криптографической подписью, каталог публикаций с поиском, избранное, а также проверка подписи для подтверждения авторства.

На заключительном этапе проведено тестирование: модульное для проверки корректности работы компонентов и фаззинг для проверки устойчивости к некорректным входным данным. Выполнена верификация продукта: подтверждено соответствие реализованной функциональности всем заявленным требованиям. Система упакована в контейнеры и развернута в облачной

среде. Платформа доступна по адресу: <https://doceum.ru>. Был настроен конвейер непрерывной интеграции и подготовлена документация.

Таким образом, все поставленные задачи успешно выполнены. Платформа Doceum готова к использованию и может применяться для создания инструкций, гайдов, методических материалов и технической документации. Проект открыт для дальнейшего расширения функциональности.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Камышева, Е.Ю. Педагогические условия реализации интерактивности в цифровой среде вуза // Вестник Шадринского государственного педагогического университета – 2022. – №1(53). – С. 30-36. – URL: <https://vestnikshspu.ru/journal/article/view/884/661> (дата обращения: 14.05.2026).
2. Витченко, О. В. Интерактивность как одно из основных требований к современным электронным образовательным ресурсам / О. В. Витченко // Международный журнал экспериментального образования. – 2013. – № 4-1. – С. 66-68. – URL: https://elibrary.ru/download/elibrary_20218792_29198281.pdf (дата обращения: 14.05.2026).
3. Cockburn, A. Writing Effective Use Cases / A. Cockburn. – Boston: Addison-Wesley Professional, 2000. – 304 p.
4. Вигерс, К. Разработка требований к программному обеспечению / К. Вигерс, Д. Битти; пер. с англ. – 3-е изд., доп. – М.: Русская редакция; СПб.: БХВ-Петербург, 2014. – 736 с.
5. What Is a Modular Monolith?: [Электронный ресурс]. URL: <https://www.milanjovanovic.tech/blog/what-is-a-modular-monolith> (дата обращения: 17.05.2026).
6. Мартин, Р. Чистая архитектура. Искусство разработки программного обеспечения / пер. с англ. – СПб.: Питер, 2018. – 352 с.
7. Richardson R. T. The Impact of Visual Design on Cognitive Load and Learning Outcomes in Digital Learning Environments [Электронный ресурс] // Journal of Online Learning and Technology. – 2014. – Vol. 10, No. 4. – URL: https://jolt.merlot.org/vol10no4/Richardson_1214.pdf (дата обращения: 17.05.2026).
8. A Gentle Introduction to WebAssembly in Rust: [Электронный ресурс]. URL: <https://dev.to/marktolmacs/a-gentle-introduction-to-webassembly-in-rust-2025-edition-1iac> (дата обращения: 17.05.2026).

9. The Twelve-Factor App: [Электронный ресурс]. URL: <https://12factor.net/> (дата обращения: 17.05.2026).
10. H2 Database Tutorial: [Электронный ресурс]. URL: https://www.tutorialspoint.com/h2_database/index.htm (дата обращения: 18.05.2026).
11. Property-based testing with React and fast-check: [Электронный ресурс]. URL: <https://dev.to/tobiastimm/property-based-testing-with-react-and-fast-check-3dce> (дата обращения: 18.05.2026).